

The Translation of Finale's Enigma File Format for CPNView

Margaret Cahill B.A.Mus.

Submitted for the award of M.Sc. in Music Technology
University of Limerick

Supervised by : Dr Donncha O Maidin

Submitted to the University of Limerick, August, 1998.

Table Of Contents

Acknowledgements	V
Abstract	VII
Introduction	1
Chapter 1. Overview of Musical Standards	2
1.1 NIFF: Notation Interchange File Format	2
1.1.1 Structure of NIFF	4
1.1.2 Temporal Information	5
1.1.3 Articulation, Ornaments and Dynamics	6
1.1.4 Other Features	6
1.2 SMDL: Standard Music Description Language	7
1.2.1 Structure	7
1.2.2 The Logical Domain	8
1.2.3 Advantages of SMDL	9
Chapter 2. Finale and the Enigma Transportable File Format	10
2.1 Finale	10
2.2 The Enigma Transportable File Format	10
2.3 General Structure of the Enigma Transportable File Format	12
2.3.1 The Entry Pool	13
2.3.2 The Entry Flag	15
2.3.3 Pitch	17
Chapter 3. Structure of the Enigma Transportable File Format	19
3.1 Clefs	19
3.2 Key Signature	19
3.3 Time Signature	20
3.4 Barlines and Repeat Signs	20
3.5 Rests	23
3.6 Articulation	23
3.7 Ties	24
3.8 Dynamics and Score and Staff Expressions	25
3.9 Grace Notes	26
3.10 Grouettes	27
3.11 Anacrusis	28
3.12 Slurs	28
3.13 Text	29
3.13.1 Lyrics	30
3.13.2 Text Blocks	31
3.13.3 Title	32
3.13.4 Instrument Names	32
3.14 Polyphony and Multiple Staves	32
3.14.1 Polyphony on Single Staves	33

<u>Chapter 4. CPNView and the Translation of the Enigma File</u>	34
<u>4.1 CPNVIEW</u>	34
<u>4.3 The Translation from Enigma to CPNView</u>	36
<u>4.3.1 Time and Key Signature</u>	36
<u>4.3.2 Clefs</u>	38
<u>4.3.3 Durations</u>	39
<u>4.3.4 Barlines</u>	40
<u>4.3.5 Anacrusis</u>	41
<u>4.3.6 Pitch</u>	42
<u>4.3.7 Articulation</u>	48
<u>4.4 Creation of the CPNView Score</u>	49
<u>Chapter 5. Testing, Further Work and Conclusions.</u>	51
<u>5.1 Testing</u>	51
<u>5.2 Further Work</u>	52
<u>5.3 Conclusion</u>	53
<u>APPENDIX 1: Program</u>	54
<u>APPENDIX 2: Test Tunes</u>	72
<u>Appendix 3 : Example of an Enigma file</u>	87
<u>Bibliography</u>	110

Acknowledgements

I would like to thank the staff of the MA/MSc in Music Technology for exposing me to an endless world of possibilities and learning and for cramming more information into a single year than I had ever dreamed possible.

A special thanks must go to Donncha O Maidin for his guidance and help in the completion of this project.

I also have to thank my parents who have supported me in every way during my year in Limerick and my disappearance into the depths of the Mac lab.

To all those who have made the Mac lab home and endured the night shift for the past year, a special thanks for ensuring that there was always time for a good laugh in the midst of all the hard work.

Abstract

The Translation of Finale's Enigma File Format for CPNView

**Author: Margaret Cahill, Department of Computer Science and Information Systems,
University of Limerick.**

A program to translate the music notation created with the software package Finale to a CPNView score. This involves the location, extraction and conversion of musical notation as represented in Finale's Enigma Transportable File Format. The translation process is implemented within the C++ framework of CPNView and provides a means of inputting music scores notated in Finale into CPNView. Documentation of the Enigma file format is given and examples of the translated files that are output from the executable program.

Introduction

Two standards have been developed and partially implemented for the computer representation of notated music. These are SMDL, an International Standards Organisation standard, and NIFF, a standard developed by the commercial software industry. A third method of representing musical scores has also developed as an offshoot of the NIFF project. The Enigma Transportable File Format was developed by Coda Music Technology for their music notation package, Finale.

The focus of this project is on the Finale's Enigma Transportable File Format and the translation of the musical notation stored in this file format to another score format. The translated score format is that of CPNView, a software environment for music representation and analysis developed by Donncha O Maidin as part the doctoral thesis "A Programmer's Environment for Music Analysis".

An overview is given of the development of the two main representation standards and their methods of representation are discussed in comparative musical categories. The general structure of Finale's Enigma Transportable File Format is given and a more detailed documentation of individual musical elements follows.

The core of the project focuses on the actual translation of this file format to a CPNView score and discusses the general approach to the task. Particular problems and difficulties that were encountered are discussed and recommendations for further research and work in the specific area are suggest

Chapter 1. Overview of Musical Standards

1.1 NIFF: Notation Interchange File Format

“NIFF is a file format designed to allow the interchange of music notation data between and among music notation editing and publishing program music scanning programs.”¹

Many such music software products existed and a new means of communication was needed to enable interaction between them. Before NIFF the DARMS (Digital Alternate Representation of Musical Scores) code was the most effective way of communicating and sharing music representations between music software. There were some problems with using DARMS, mainly that it is not competent enough at representing information relating to the graphical content of written music. Although SMDL was instigated to be a standard for music representation, commercial companies felt that it did not meet their needs and so a group of companies involved in producing music software decided to come together to devise a standard for music representation that would be specifically designed around their requirements.

In early 1994 delegates from the following companies met to begin the development of NIFF:

Passport Designs	--- publishers of Encore
Coda Music Technology	--- publishers of Finale
San Andreas Press	--- publishers of Score
Musitek	--- publishers of MIDISCAN
Cindy Grande	--- publishers author of NoteScan
Nicholas Carter	--- publishers author of SightReader

¹ p.491 Beyond MIDI – The Handbook of Musical Codes Cindy Grande

At the start of 1995 NIFF 5 was released and at about the same time Coda Music Technology decided to withdraw from the group to concentrate on devising a file format of their own - the ENIGMA transportable file format. They were replaced in the NIFF development group by :

Mark of the Unicorn	---publishers of Mosaic
Opcode Systems	---publishers of Overture
Twelve Tone Systems	---publishers of Cakewalk
Musicware/TAP Music Systems	---publishers of Nightingale

Further work on the file format led to the release of version 6a in August 1995 and the current version available is NIFF version 6a.1.

NIFF files are based on the structure of Microsoft's RIFF file format (Resource Interchange File Format). One of the most important features of this is that items of data that are related to one another are linked together into entities known as chunks. These chunks can then be grouped together into lists. The significance of this is that the file format is extendable and can be enhanced in the future, which was one of the main aims of the NIFF developers. Another important aim was that the file format was to be non-proprietary and platform independent, which is achieved by the file formats relationship with RIFF. The combination of these features has created a flexible and potentially very successful file format. It has even been said that :

“NIFF is to MIDI what formatted word-processing files are to ASCII files.”²

² <http://www.cs.princeton.edu/~dgslomin/nifty/paper/index.html> Donald G. Slomin

1.1.1 Structure of NIFF

It has become somewhat standard in the area of computer music representation to divide the data about the music into different components.³ In order to represent music successfully one needs to have logical, graphical and performance information. NIFF developers have divided the graphical information into data that is related to page-layout and that which is not related to page-layout. NIFF does not require all these kinds of data to function but logical information is essential.

The structure of a NIFF file is organized in a hierarchical fashion in “page-ordered format”⁴. Pages incorporate systems, which in turn consist of staves which are made up of time-slices and individual symbols that represent music.

The file itself consists of a setup section and a data section. The setup section is comprised of header data including default information and definitions. The data section involves the actual code that represents the music notation.

A major advantage of NIFF is its ability to keep the file and data required as small as possible while allowing for expansion and the possibility that large amounts of information may be involved at a later stage. This is a result of the system of defaults and the possibility that these may be overridden, for example, in the setup up section a default may be defined as staff 1 = horn 1. If the horns play homophonically later in the piece of music they may need to be represented on one single staff and so the default will need to be overwritten so that staff 1 =

³ p.493 Beyond MIDI The Handbook of Musical Codes Cindy Grande

⁴ p.497 *ibid.*

horn 1 and 2 or staff = horn 1, 2, 3 and 4.

The relationship between musical symbols is important in NIFF. The way in which these relationships are described is crucial to the flexibility and effectiveness of the file format. If the positioning of a symbol or chunk depends on the location of another it is said to be dependent. The entity is dependent on the anchor. Dependent symbols can be related to more than one anchor as might be the case with beaming of notes. Symbols such as ornaments, articulations and accidentals are assigned chunk types particular to them.

Tags are an important part of NIFF. They help to ensure that the file stays as small as possible. Specifically they are data items that are used to add extra components to the essential part of chunks. In this way the chunk only keeps enough space for the options specified and not for every possible music scenario that could occur. Software developers can define chunks and tags to suit their individual needs and thus customize the file format.

1.1.2 Temporal Information

Temporal data in NIFF leads to two different options for playing the music represented in the file. The first involves strict playback of the music, with the exact notation being used. The second option allows playback of the music with an element of Rubato, giving a more human-like performance. Cindy Grande in her description of these playback timing options uses the terms “logical” and “performance” playback. For each the start time and duration of each note of the piece are required. Where the “performance” playback is chosen the start times and durations of the notes are represented as offsets from the logical time data and are measured by MIDI clocks.

In NIFF you can have any number of beats in a bar and you do not have to adhere to logical time signatures. This is directly related to another of the developers aims - that no elements would be viewed as being illegal and that if a program was reading a NIFF file it would ignore or adequately deal with elements it could not recognize.

1.1.3 Articulation, Ornaments and Dynamics

A chunk is associated with each type of articulation, ornament, and fingering etc. The articulation chunk defines various types of accents, staccato, bowing indications etc. and the ornamentation chunk defines such objects as trills, mordents and turns.

The dynamic chunk defines dynamics in their text form. The dynamics range from “pppp” to “ffff” and “sp”, “sf”, “sfz”, “fz”, “fp”, “cresc.”, “crescendo”, “dim.”, and “diminuendo” are also included.

1.1.4 Other Features

NIFF allows for the representation of Early music (unmeasured music and four-line staff) and modern music with unusual meters, effects etc.

A NIFF Software Developer’s Kit (SDK) has been written by Tim Butler. This includes software libraries and tools that help in writing and reading NIFF files and also sample files of NIFF code. This SDK is available from the NIFF download site at “<http://mistral.ere.umontreal.ca/~belkina/N>”, which also contains the current specification , header files and other documents relating to NIFF. This is a directory of the homepage of Alan Belkin, the technical coordinator of NIFF.

1.2 SMDL: Standard Music Description Language

SMDL is an application of HyTime- "Hypermedia/Time-Based Structuring Language".

HyTime is itself an application of SGML - "Standard Generalized Mark-up Language", an international standard. HyTime

"was invented to offer a standard for accomplishing the creation, life-cycle maintenance, and even rendition of hypermedia documents".⁵

HyTime extends the functionality of SGML to allow the representation of video, moving pictures and sound. SMDL uses many of the features of HyTime with special features that relate directly to music representation. SMDL

"defines a language for the representation of music information, either alone, or in conjunction with text, graphics, or other information needed for publishing or business purposes".⁶

HyTime uses an object-orientated structure. Data is arranged as a set of hierarchies and the means are provided for addressing data, linking between objects and can represent temporal aspects of position and duration - statically (in score) and in performance. SMDL, as an application of HyTime inherits these.

1.2.1 Structure

Like NIFF, SMDL arranges the data representing music into different components

⁵ p.159 Readme1st-SGML for Writers and Editors Ronald turner, Timothy Douglas, Audrey Turner

⁶ <http://www.student.brad.ac.uk/srmounce/smdl.html> Steve Mounce

which it calls domains. The most important of these is the logical domain (known as the cantus). The other domains involved in SMDL are visual, gestural, and analytic.

The logical domain deals with abstract musical information such as pitch and duration. The visual domain uses the HyTime mechanism of linking which allows the linking to an external file format. The external format could be an encoded music file itself (for example in DARMS or NIFF) which contains information about graphics.

The gestural domain allows for links to musical objects in the cantus and thus can represent performance. The analytic domain again allows linking to the cantus so that analysis and commentary about a piece are included.

1.2.2 The Logical Domain

A clear description and implementation of the cantus is required in the logical domain as the fundamental representations occur here.

The pitch of a note can be represented as a specific frequency, by a lookup table (gamut) which gives the name of notes or as a function of a reference frequency.

There are two different ways of measuring duration in SMDL. The first involves real-time where duration is measured in seconds and minutes and the second involves the use of virtual time units (VDUs). (Virtual time is like common music notation in which the length of a note is affected by the meter, tempo, etc.)

A description of types of articulation or ornamentation can be held in a lookup table as with pitch or separately. The visual representation of these symbols or the text that represents it

may appear in the score, for example, a staccato may be represented by the word “staccato” or by the presence of “.”. Gestural information also exists about how the articulation or ornament affects the original note. This information may be kept in a style sheet that indicates how a particular symbol should affect a note.

Data related to dynamics is stored in a similar way to articulation. The name or symbol for a dynamic appears in the score. The interpretation of information is left to performer.

Timbral information is represented in SMDL by the use of threads, which represent different instrumental lines. The instrument for each thread is defined at the beginning of the code.

SMDL code is more complex than music representation codes such as DARMS, Humdrum Kern, and EsAC but it is not expected to be seen directly by the user, only by the software developers. It is envisaged that a user friendly interface and software tools will be developed so that the code could be manipulated without the user being exposed directly to the code.

1.2.3 Advantages of SMDL

SMDL is flexible. It can contain other types of notation, by linking to external file formats. It combines information about performance in a format which differentiates from between the fundamental aspects of music, such as pitch and duration, and information that relates to performance. Many music representation languages exist today. Translation is often needed to allow one file format to be made readable by another. SMDL standard means that if

music notation codes are translated to SMDL it reduces greatly the need for many separate translators and converters.

Chapter 2. Finale and the Enigma Transportable File Format

2.1 Finale

“Finale is a powerful program for music transcription, notation, playback, and publishing”⁷.

Finale is one of the most popular software programs that deals with notating music and is available in versions for Windows and Macintosh operating systems. The current release of Finale is Finale 98.

Finale allows the user to have a lot of control over the graphical representation of the musical score such as width of bars, stem length, thickness of lines etc. However, the user does not have to be exposed to decisions such as these and if preferred Finale’s built in system of defaults can be used and only strictly musical elements created and altered.

The version used in undertaking this project was Finale 3.0 for Windows.

The Macintosh equivalent was also consulted as was Finale version 3.5.1 for Power Macintosh.

2.2 The Enigma Transportable File Format

When Coda Music Technology withdrew from the NIFF development group they did so in order to concentrate on the development of a new file format of their own. They decided that supporting NIFF would not be commercially viable and used their resources to develop the Enigma Transportable File instead. When saving files in Finale, among the file format options

given is The Enigma Transportable File format with the extension “.etf.”

On 2/8/96 Coda Music Technology released documentation on ETF. This documentation identifies only one of the important sections in an ETF file. The documentation given refers to the entry pool, which forms only a tiny segment of a file of this type and only some information about the representation of the notated music is given here. The most obvious details that are missing from the documentation are those related to time and key signatures. The representation of other musical features such as the clef used in the music, dynamics, articulation, existence and types of barlines, text information such as titles and composers is totally ignored in the Finale documentation. When Coda Music Technology released this documentation they invited questions and input from Finale users about the contents and layout of the file format, so they could gauge the need for data about the file and release further documentation as required. However on requesting information from the technical support at Coda Music Technology they reported that no further documentation would be released. The reason stated for this was :

“This is due in large part to the fact that no such a document currently exists: its creation requires that our engineers take time away from their coding duties to document information that is buried in the code itself”.⁸

This meant that the information relating to music had to be identified and extracted from the Enigma file before the translation process could begin. This was an aspect of the project that had not been anticipated as preliminary research had indicated that Coda Music Technology

⁷ p.vii Finale The Art of Music Notation Volume 1 – Installation & Tutorials

⁸ Scott Yoho, Coda Music Technology in an e-mail reply to request for information from Finale about the Enigma file format.

would be willing to release information and answer questions relating to their representation form.

In order to find the lines in the code that represented various musical elements, various Finale files had to be created for each possible musical scenario and the notation saved as ETF files. These files then had to be opened in a text editor in order to view the code and searched through to find the changes that occurred from one key to another. As the Finale default file (a bar in C major, in common time, with a semibreve rest) produces approximately twenty pages of text this was a very time consuming task and meant that much of the time scope for the project was spent researching and translating the Enigma representation.

2.3 General Structure of the Enigma Transportable File Format

Enigma files can store information not only about the more fundamental aspects of music notation, such as pitch, duration and metre but can also record graphical information about the visual presentation of the score and any performance aspects of a piece of music that have been specified in Finale. The bulk of an Enigma file seems to be concerned with graphical details and details that are concerned with the platform that the software is running on. The information that is relevant to the actual notated music itself proves to be a small proportion of the overall file size. The difficulty in extracting the relevant information lies in the fact that this data is scattered throughout the file and related data items are not always located in close proximity to each other.

An ETF file is divided unevenly into three sections - the “header “, “others “, and “details”. The name of the file format, “ENIGMA TRANSPORTABLE FILE”, is stored at the top of the file. All lines in the file are prefixed by the “^” symbol and this is generally followed by a combination of characters and bracketed numbers, which I will call a tag, for example, “**^MS(1)** 600 0 4 1024 4 16 ”. The opening “header” section contains information about the version of Finale being used and the year of it’s release. The next section “others” is the largest in the file and the lines here are usually comprised of a tag followed by four to six numbers, except where the information is printed text as with a title. The code representing features such as key and time signatures, articulation, dynamics, title, type of barlines and repeats, slurs can be found in this section.

2.3.1 The Entry Pool

One of the most important sub-sections is found under the “details” heading and the developers refer to it as the “entry pool”. This section is located at the end of an ETF file and is only followed by lyrics or other text if they occur. The entry pool contains the pitch and duration information, as well as many bit flags that indicate the existence of certain musical attributes, the details of which can be found in other parts of the file.

The entry pool consists of one or more entry records, which are followed by one or more note records if the entry is a note. The entry pool generated by a scale of C major is shown below. (The scale starts at middle C and all the notes are crotchets)

^entries

^eE(1) 0 2 1024 0 \$C0000800 0 1	(entry record)
0 \$80000000	
^eE(2) 1 3 1024 0 \$C0000800 0 1	
16 \$80000000	(note record)
^eE(3) 2 4 1024 0 \$C0000800 0 1	
32 \$80000000	
^eE(4) 3 5 1024 0 \$C0000800 0 1	
48 \$80000000	
^eE(5) 4 6 1024 0 \$C0000800 0 1	
64 \$80000000	
^eE(6) 5 7 1024 0 \$C0000800 0 1	
80 \$80000000	
^eE(7) 6 8 1024 0 \$C0000800 0 1	
96 \$80000000	
^eE(8) 7 0 1024 0 \$C0000800 0 1	
112 \$80000000	

The entry record consists of an entry tag (“^eE(number) “), which indicates the entry number, and seven entry fields. The information contained in the entry record relates to the entire entry i.e. it affects all the note records that follow. These entries are kept on a doubly linked list and this is shown in the first and second fields of the entry record. The first field gives the number, if any, of the previous entry and the second field indicates the number, if any, of the next entry in the file. The third field in the record gives the duration of the entry, ie. the length of the note, chord or rest. The duration of the entries is measured in Enigma Duration Units (EDUs), with a crotchet corresponding to 1024 EDUs. All other durations can be calculated in relation to this:

demisemiquaver (thirty-second note) =128

semiquaver (sixteenth note) = 256

quaver (eighth note) = 512

crotchet (quarter note) = 1024

minim (half note) = 2048

semibreve (whole note) = 4096

The next field gives information about the manual positioning in Enigma Virtual Page Units (EVPUs), which is set at 288 per inch.

2.3.2 The Entry Flag

The fifth entry field is known by Coda Music Technology as the “entry flag”. It is signified by the symbol “\$” which is not part of the data itself. This is one of the sections in the file format where the developers have extensively used bit flags to represent data about the notated music. This flag is a 32-bit hexadecimal number that gives information about the entire entry. A list of some of the most frequently used and hence important bit flags follows

x80000000 this bit is always set to indicate that the entry is “legal”

x40000000 this bit is set if the entry is a note, and not set if entry is a rest⁹

x04000000 indicates use of a chord symbol record

x01000000 indicates floating rests (those rests not positioned manually)

x00800000 indicates that the current note is a grace note

x00200000 indicates the use of articulation on this entry

x00100000 indicates the existence of lyrics

x00080000 indicates the existence of a groupette

⁹ Hexadecimal C indicates that an entry is a note and that it is legal.

x00000800 flag used to determine beaming

The sixth field in the entry field is the “extended entry flag”. This is a 16-bit number that is represented in the file as a decimal value. Some of the main extended entry bit flags are:

x0001 indicates the presence of a staff assignment record
x0008 indicates that the entry is part of a groupette
x0004 indicates that the entry is tied from an entry in the
previous bar

The last field in this record indicates how many note records follow. The maximum number of notes in an entry is twelve. If the entry is a rest this field is set to zero.

2.3.3 Pitch

One or more note records can follow the entry record. The note record consists of a “TCD” (Tone Centre Displacement), and a note flag. The TCD is a 16-bit value that is divided unevenly into two parts. The low order nybble stores what is known as the “alteration” of the note, as a signed value. The other twelve bits of the TCD give the pitch and these four bits tell whether the note is natural (0), sharp (1), double sharp (2), flat (-1), or double flat (-2).

Microtonal and non-Western art music can utilize the further alterations possible from the use of four bits, where it is possible to record an alteration of -8 to +7.

The pitch of the note is recorded in the other twelve bits of the TCD. The developers refer to the pitch here as the “harmonic value” of the note, as the pitch is defined in relation to the key signature given. The values given correspond to the diatonic pitch of the given key, for example, in the key of C major the harmonic value of C =0, D = 1, E = 2, F = 3, G = 4, A = 5, B = 6, and C an octave higher than the tonic = 7. Any alterations, i.e. accidentals, to the diatonic scale are recorded in the low order nybble. In an ETF file a chromatic scale of C would be represented as follows:

C3	-112	C4	0	C5	112
C#3	-111	C#4	1	C#5	113
D3	-96	D4	16	D5	128
D#3	-95	D#4	17	D#5	129
E3	-80	E4	32	E5	144
F3	-64	F4	48	F5	160
F#3	-48	F#4	49	F#5	161
G3	-63	G4	64	G5	176
G#3	-47	G#4	65	G#5	177
A3	-32	A4	80	A5	192
A#3	-31	A#4	81	A#5	193
B3	-16	B4	96	B5	208

(Referring to C4 as middle C)

The note flag is similar in structure to the entry flag. It gives information about individual notes, or notes of a chord. Some of the most important bit flags are as follows:

x80000000	this bit must be set, indicates legality
x01000000	indicates the existence of an accidental
x40000000	tie starts on this note
x20000000	tie ends on this note

Chapter 3. Structure of the Enigma Transportable File Format

3.1 Clefs

The information concerning clefs in Enigma files is found in the lines beginning “^GF” which is situated in the “^details” section. A line of this type occurs for every bar of music notated. The tag is followed by the stave and bar numbers enclosed in brackets. The rest of the line is made up of 5 numbers, the second of which indicates the type of clef present on this stave, at this particular bar. The numbers representing the clef types are as follows:

Number	Clef
0	Treble
1	Alto
2	Tenor – F
3	Bass
4	Percussion
5	Tenor – G
6	Contrabass
7	Baritone

3.2 Key Signature

The information representing the key signature of the piece can be found in the line beginning “^MS”. One of these lines exists for each bar of the music and the bar number is given in round brackets after the tag. There are six numbers in this line and the second indicates the key signature¹⁰ of the piece, as shown below:

¹⁰ When referring to key signature in this context I mean the number of flats or sharps that occur. The key names given indicate the presence of these but do not indicate tonal musical keys as such.

No	0	1	2	3	4	5	6	7	255	254	253	252	251	250	249
Key	C	G	D	A	E	B	F#	C#	F	Bb	Eb	Ab	Db	Gb	Cb

3.3 Time Signature

The time signature of a piece of music can also be determined by this line of the file.

The number of beats and the duration of these beats is given. The duration is recorded in Enigma Duration Units(EDUs), as mentioned in section...XXX. These are the third and fourth numbers in the “^MS” line, for example, common time is represented as 4 beats each of a crotchet duration (1024):

^MS(1) 600 0 4 1024 4 16

In situations where the number of beats does not correspond directly to the time signature, for example in 6/8 there may be 6 quavers in a bar, but there may also be 2 dotted crotchet beats in a bar in which case the time signature will be represented by the numbers 2 1536.

3.4 Barlines and Repeat Signs

These musical features are represented in the line beginning ^MS. This tag is followed by a number in brackets which indicates the bar that the information refers to. The last number in this line represents the type of barline or repeat symbols used as shown below.

<u>No in ETF</u>	16	8224	8256	8272	8240	8192
<u>file</u>						
<u>Type of</u>	Normal	Double	Solid	Final	Dashed	Invisible
<u>barline</u>						

The numbers representing the various barlines change if a dynamic marking or “Smart Shape”¹¹ was added to the bar in question. If a symbol of this kind is encountered in a bar the numbers representing the barlines change to :

<u>No in ETF file</u>	16400	24608	24640	24656	24624	24576
<u>Type of barline</u>	Normal	Double	Solid	Final	Dashed	Invisible

The presence of graphic and text repeats is also indicated by this particular number. The presence of a text repeat such as a “DaCapo” or “Fine” marking is merely noted here while graphical repeat lines are represented here as indicated:

No in ETF file	24	20	22	18	17
Musical meaning	right facing repeat	left facing repeat	first repeat bar(s)	second repeat bar(s)	text repeat in this bar

As is the case with barlines, these numbers are altered if a dynamic marking or “Smart Shape Entry “ appears in the same bar. The numbers representing this are:

No in ETF file	16408	16404	16404	16402	16401
----------------	-------	-------	-------	-------	-------

This representation feature meant that I had some difficulty in extracting this information from the Enigma file format. I found that the bit structure of the original number was similar to the number that represented the existence of the dynamic or “Smart Shape” symbol and so using bitwise operators I was able to determine if particular bits that formed a number were set or not.

¹¹ A “Smart Shape Entry Palette” is available in Finale which includes symbols such as slurs and dynamics

The text repeats supported by Finale are:

D.C. al Fine
D.S. al Fine
To Coda #
D.S. al #
Fine



Go to Measure #
D.S. al Coda
D.C.al Coda

If a text repeat of this kind is present in the music the last number in the line with the “^MS” tag is set to 17. There are always ten lines with an “^RU” tag and ten with an “^RT” tag in an Enigma file, each with the numbers 1-20 following in brackets. The former line indicates which type of repeat is used and the latter line of code contains the actual text that is printed in the music. The first of these lines refers to the first type of repeat in the list above, “D.C.al Fine” and the second to “D.S. al Fine”, and so on. If the first number in the “^RS” line is set to zero it indicates that there is no text repeat of this type present. If it is set to other than zero a text repeat of the relevant type is present in this bar. If the text repeat “D.C. al Fine” is present in bar 4 the relevant lines of the file might be: (not in order)

```
^MS(4) x   x   x   x   x 17
^RS(1) 345 x   x   x   x   x
^RT(1) "D.C. al Fine"
```

The ten “^RS” and “^RT” are always present in a file and it is the combination of the change in the “^MS” line to 17 and the alteration of the first number in the “^RS” line that indicates that this section of the code is relevant.

3.5 Rests

If an entry is a rest an entry record will exist for it but there will not be a note record, as there is no note to record. The last number in the entry record indicates how many if any notes are to follow and if this number is set to zero it indicates that the entry is a rest. The first bit in the entry flag is used to indicate the presence of “floating” rests. Most rests are considered as “floating” rests because they are not manually placed in position on the stave. Thus an entry record for a semibreve rest might be:

`^eE 0 1 4096 0 $81000000 0 0`

The first bit in the entry flag is set to 8 if the entry is legal and this bit is also set to 4 if the entry is a note but not set if the entry is a rest. This effectively means that if an entry is a note the first bit of the entry flag will be set to hexadecimal C and if it is a rest it will be set to 8.

3.6 Articulation

The third bit in the entry flag indicates the presence of an articulation symbol if it is set to 2: `$C0200000`. The information about the actual type of articulation symbol used is contained in a line in the “details” section. This line, beginning with the tag “`^IM`” is not present in the code unless articulation is used. The relevant numbers in the line are highlighted below

`^IM(0,1) 1 6 0 1 6 0`

The second number in the brackets indicates which entry number the articulation refers to and the first number in the line proper indicates the type of articulation. The numbers correspond to the choices of articulation given in Finale as shown:

Articulation Selection

1 	(1)	2 	(2)	3 	(3)	4 	(4)
5 	(5)	6 	(6)	7 	(7)	8 	(8)
9 	(9)	10 	(10)	11 	(11)	12 	(12)
13 	(13)	14 	(14)	15 	(15)	16 	(16)

17 	(17)	18 	(18)	19 	(19)	20 	(20)
21 	(21)	22 	(22)	23 	(23)	24 	(24)
25 	(25)	26 	(26)	27 	(27)	28 	(28)

29 	(29)
---	------

3.7 Ties

Two bits in the note flag of the entry pool are set to indicate the start and end of a tie.

The first bit of the flag is set to 4 if a tie starts on the entry and it is set to 2 if a tie ends on a note. When this is combined with the bit that indicates a legal entry (8) the bit settings become hexadecimal C and A respectively.

The extended entry flag is set to 4 if a tie is held over from a previous bar.

3.8 Dynamics and Score and Staff Expressions

Dynamics can be inserted into music in Finale by choosing the “Score Expression Tool” or “Staff Expression Tool”. The latter attaches the chosen dynamic to the staff chosen while the former is used to refer to all staves used. Obviously, when dealing monophonic music with only one staff either tool can be chosen for including dynamics.

When using the “Score Expression Tool” new lines are inserted into the code, in “others” section. The tag for these lines is “^DY” and this is followed by a single number in brackets that indicates the bar number that the expression is present in. The first subsequent number indicates which dynamic or expression is used. The dynamic markings supported by Finale and their corresponding Enigma file numbers are:

f	1
mf	2
ff	3
fff	4
ffff	5
p	6
mp	7
pp	8
ppp	9
pppp	10
sfz	11
sffz	12

Other musical expressions that are chosen from this selection and the numbers that represent them are:

Adagio	13
Moderato	14
Allegro	15
8ve	17
8vb	18
accel	19
rit	20
A	21
B	22

C	23
D	24

An “fff” marking in bar three would be stored as: “^DY(3) 4 “. A further five numbers make up this line of the file, but I have not successfully translated these numbers yet and have not been able to determine exactly which note of the bar the symbol is attached to.

If the “Staff Expression Tool” is used instead of the “^DY” tag, new lines with the tag “^ED” are included in the file for each expression used. The numbers representing the various expressions are identical to those listed above. The tag is followed by two numbers in round brackets that represent the stave number and bar number, for example, an “fff” marking on the first stave in bar 1 would be: “^ED(0,1) 4 “. The extended entry flag is set to 1 if the “Staff Expression Tool” is used.

3.9 Grace Notes

Grace notes can be created in two different ways in Finale. The first method using the “Simple Entry Palette” is recorded in the Enigma files by setting bits in the entry flag. The third bit is set to 8 to indicate that the entry is a grace note and the last bit is set to 1 if the grace note stem is an up stem and 0 if it is a down stem (\$C0800001 or \$C0800000). The pitch of the grace note is recorded in the TCD of the note record that follows.

The other method is to choose grace notes from the selection of articulation symbols. If this method is used the grace note does not show up in the file as above but instead, as with other articulations, the third bit in the entry flag is set to 2 and the number in the “^IM” line indicates that the articulation is a grace note.

3.10 Groupettes

The existence of groupettes are indicated by setting bits in the entry flag and the extended entry flag. In the entry flag the start of a groupette is defined by the bit: \$C0080000.

The extended entry flag is set to 8 for each entry that is part of a groupette.

Detailed information a groupette is stored in the lines that follow the tags “^TN” and “^TP” in the “details” section. These lines do not ordinarily occur in the file, but only exist if groupettes are used. The latter line contains the information needed about the duration of the groupette.

The second number that occurs in the brackets states the entry number that the groupette begins on. Five further numbers make up a line of this type. This line indicates how many notes of a particular duration make up the groupette and the actual resolves duration of the groupette. A quaver triplet on the first entry would be represented by the line:

“^TP(0,1) 3 512 2 512 3”

This shows that 3 beats of duration 512 make up the groupette and that the actual duration of the groupette takes up the space of 2 beats of duration 512.

Similarly the code to represent a quadruplet of crotchet values is:

“^TP(0,1) 4 1024 2 1024 3”

3.11 Anacrusis

The presence of an anacrusis and its duration can be determined by the line with the tag “^17” near the start of the “others” section. The last number in this line represents the duration, if any, of an anacrusis. The line generally appears as :

“^17(65534) 0 0 0 -200 6 0 “

if there is no anacrusis and

“^17(65534) 0 0 0 -200 6 1024”

if an anacrusis of duration 1024 was present in the notation.

3.12 Slurs

The representation of slurs in the Enigma file is more complex than the representation of many other musical features, as a number of lines of code are involved to denote a single slur. Four of these lines with the tags “^aX”, “^cX”, “^eX”, “^oX” are grouped together. The fifth line, with an “^sX” tag is included further down in the code. Each of these tags is followed by a bar number in brackets, indicating the number of the slur - the first, second or subsequent occurrences of a slur. There are six numbers in each line proper and the second of these is used to indicate if the slur is placed above or below the notes concerned, zero indicates that the slur is below the notes, and 1 that it is above the notes. The “^aX” and “^oX” lines are generally made up of six zeros, indicating that the required information is contained in the “^cX” and “^eX” lines. The first number in the “^eX” line indicates which bar the slur begins in and the

corresponding number in the “^cX” line indicates the bar in which this slur ends. The “^sX” line is identical to the “^cX” line. For each slur present in the music two lines with the tag “^DY” are also used to represent the slur. These lines were discussed in the section on dynamics and their use in this instance is unrelated to that above.

The lines of code mentioned here are used to represent a single slur and if more than one slur is used multiple instances of lines of this type will occur. It has not yet been determined from the code which exact notes the slurs refer to, only the bar in which a slur starts and ends.

The problem with extracting information about slurs from Enigma files is demonstrated by the fact that Finale themselves actually changed the way in which they represented slurs in recent releases of their product.

In version 3.5.1 none of the lines of code discussed in this section are used and slurs are represented by the setting of particular bits in the extended entry flag, and the code in four lines with the tag “^Sx”, in the “others” section and two others with the tag “^Ex”, in the “details” section. The exact notes included in the scope of a slur has not yet been determined. The extended entry flag in this version is set to decimal 128 to represent a normal note and to 160 to represent a note with a slur.

3.13 Text

Finale allows for the inclusion of many different types of text, including song lyrics in the music itself and other text such as instrument names and composers that are not integral to the music.

3.13.1 Lyrics

Finale allows the user to specify verses, choruses and section when adding lyrics to a notated piece of music. The third bit in the entry flag is set to 1 to indicate that lyrics have been assigned to that particular entry. The text for the lyrics is recorded in the “^lyrics” section of the Enigma file, which is found at the end of a file and 512 instances of each type of lyric is allowed in a single file. If verses are defined the tag “^verse” can be found after the “^lyric” tag and different verses are distinguished by a number that follows the “^verses” tag in rounded brackets, for example, if two verses were present the structure of the code would be as follows:

```
^lyric^verse(1)verse 1text ^verse(2)verse 2 text
```

These lyrics can be entered in two different ways when using Finale, by typing straight into the score or by the “click assignment” method which allows the user to type all the verse into a window and then click on the relevant notes under which the text should appear. A new set of lines appear in the “^details” section of the code for each instance of a verse, section and chorus, (“^ve”, “^se”, “^ch”). Each of these tags is followed by two numbers in brackets, the second of which indicates which entry number the word/syllable is associated with. These lines have five more numbers. The first is the verse, section or chorus number that the lyrics belong to and the next is the number in instances of the word/syllable. The code for a single staff with a verse eight words long, followed by a chorus six words long would be:

```
^ve(0,1) 1 1 0 0 0
```

```

^ve(0,2) 1 2 0 0 0
^ve(0,3) 1 3 0 0 0
^ve(0,4) 1 4 0 0 0
^ve(0,5) 1 5 0 0 0
^ve(0,6) 1 6 0 0 0
^ve(0,7) 1 7 0 0 0
^ve(0,8) 1 8 0 0 0
^ch(0,1) 1 1 0 0 0
^ch(0,2) 1 2 0 0 0
^ch(0,3) 1 3 0 0 0
^ch(0,4) 1 4 0 0 0
^ch(0,5) 1 5 0 0 0
^ch(0,6) 1 6 0 0 0

```

The actual text for the lyrics would follow the “^lyric” tag and the code shown above, along with the bit in the entry flag, would indicate which syllables and words referred to which note entries.

3.13.2 Text Blocks

Blocks of text that are not lyric related can also be added to the musical score. This could be directions to a performer or other personal text that is not an intrinsic part of the music. When creating a text of this kind in Finale each instance is given a unique ID. This text appears in the Enigma file under the “^lyrics” section, following it’s “^text” tag. The ID of the text group is printed in round brackets before the actual text and the last number in the line with the “^MS” tag (discussed in relation to time and key signatures) is 272. New lines of code with the tags “^PT” (in the “^others” section) and “^mt” (in the “^details” section) appear when text of this nature is used. A single number in round brackets follows the “^PT” tag and this tells which bar the text occurs in. The line itself is comprised of six more numbers and the first of these corresponds to the ID number that is printed before each block of text at the bottom of the Enigma file.

3.13.3 Title

The title of a piece of music appears in a group of lines in the “^others” that each have the tag “^HT”. Each of these lines is followed by a pair of inverted commas, and some of these will contain text if a title is given to the piece. The maximum number of characters allowed in the inverted commas is twelve and so if the title has more than twelve characters it will be split into two or more and appear over subsequent lines of code. If a title is specified it is represented in the first line with the tag “^HT(1)” which follows a number of “^HT(0)” tags. If the title tool is used to add other text that would appear at the top of a score such as the name of the composer or performer that text is included in one of the lines with the tag “^HT”.

3.13.4 Instrument Names

Instrument names and abbreviated instrument names can be associated with staves in Finale through the use of “Staff Attributes”. The instrument name is printed in the Enigma file by a line with the tag “^IN”. The number of the staff that the instrument name applies to is given in brackets after these characters. Abbreviations of the instrument name, if created in Finale, are given further down in the file in the line with the tag “^in” and the staff numbers follow as with the full name length version above.

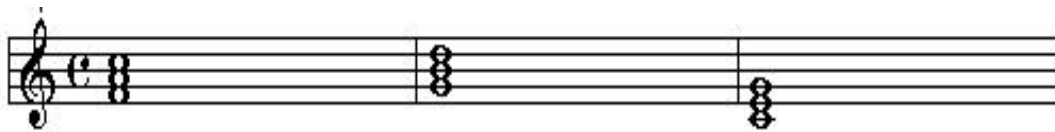
3.14 Polyphony and Multiple Staves

Although monophonic music has been concentrated on here, Finale allows for the notation of chords and polyphony on single staves, as well as scores that contain multiple staves

of music.

3.14.1 Polyphony on Single Staves

Finale allows the user to notate up to twelve notes in a chord on a single staff and eight independent lines of music. The number of notes in an entry is defined in the entry record, of the entry pool. The last number in this line indicates how many note records follow and any information and bits set in the entry record apply to all the note records that follow. The notated appearance of three chords in C major is shown below and this is followed by the representation of the chords in Enigma file format.



```
^Ee(1) 0 2 4096 0 $C00000800 0 3
      48 $80000000
      80 $80000000
     112 $80000000
^Ee(2) 1 3 4096 0 $C0000800 0 3
      64 $80000000
      96 $80000000
     112 $80000000
^Ee(3) 2 0 4096 0 $C0000800 0 3
      0 $80000000
     32 $80000000
     64 $80000000
```

Polyphony on a single staff is implemented in Finale by layers of voices, with the ability for each layer to have independent rhythm and stems. Due to time constraints the notation of polyphonic music in Finale has not been dealt with.

Chapter 4. CPNView and the Translation of the Enigma File

4.1 CPNVIEW

“CPNView is a score representation which is designed to facilitate writing of algorithms to process common practice notation scores.”¹²

CPNView is concerned with abstractions of common practice notation scores. One of the main uses for CPNView is in the analysis of music scores. CPNView operates in a C++ object-oriented environment and the classes and objects representing elements of musical scores have been constructed within the framework. For the most part the user will call “member functions” of these classes to carry out tasks.

Classes exist within CPNView to represent musical elements such as barlines, durations, pitch, notes, rests as well as more analytically based classes such as “PitchClasses” which represents pitch class sets. These classes are accessed through a “ScoreIterator” class which traverses a monophonic or polyphonic “Score” and allow the user to extract information about the particular music being used and processing of this information.

At the moment CPNView is not concerned with graphical details such as page layout and stem lengths. A lot of information that was contained in the Finale Enigma files could be ignored as it wasn’t relevant to CPNView.

4.2 The Approach to the Translation Process

The program to translate the Enigma files to CPNView score format is structured so that the extraction of the representation of each musical element from a file is contained within a function. The method used to isolate each line of relevant code was to send the String str to each function where a StringIterator would then be created. Using member functions related to CPNView's StringIterator, a particular combination of characters (a tag), could be searched for. The most frequently used of these was the "heads" object which enabled a specific to be located. When a particular tag was located, that line of code could be stepped through until the numerical value relevant to the task was encountered. Some cautionary coding elements have been incorporated into the program so that if data in a particular location of the file was not the type that was expected, and therefore could not be translated, the program would exit the function and return to main(). CPNView variable types for musical elements were defined in main() and passed to the relevant function where the value representing this particular musical feature in the Enigma file was located and this value was assigned to the CPNView type.

When the necessary elements of a musical score had been located and converted to CPNView variable types, the CPNView score was then created by arranging these variables in score format.

The order in which the data occurs in an Enigma file does not always correspond to the

¹² pp.112 CPNView-A Programmer's Environment for Music Analysis Donncha S. O Maidin

order of a musical score, for example, the placement of the information relating to the time signature of a piece is located before the data that represents the clef type of a stave, even though the clef appears before the time signature in a traditional musical score. Where multiple instances of an element may occur in a score, such as barlines and notes, the data representing each type will usually be grouped together in the Enigma file and not laid out in the order of a conventional score, where a number of notes would be followed by a barline. The implication of this was that some information needed to be stored until another connecting piece of information was located and converted, at which point the translated score could be compiled in order. A lot of the lines of code in an Enigma file could be ignored as they did not directly relate to music representation. Code representing graphic details and platform specific information etc. constitute a large part of the Enigma file format and these could be ignored as the information needed to create a CPNView score is strictly music related.

Each function in the program consists of a framework of using a ScoreIterator to step through irrelevant information in the Enigma file until the data required by the function is located. The steps of the framework have been left in place so that further work on the project may be facilitated. Many numerical values have not been stored as they were not directly relevant to the task at hand, but the structure for locating much more information is in place and if needed they could be simply assigned to variable types.

4.3 The Translation from Enigma to CPNView

4.3.1 Time and Key Signature

The number in the Enigma code representing key signature was assigned to a variable which was then checked against an array of possible key signatures. This array was defined as CPNView enumerated type “keySigType” and the number representing key signature that was extracted from the ETF file was checked against those present in the array. When the number was found in the array the corresponding element number in the array of CPNView key signatures was stored as the key of the music (as a variable type keySigType).

```
int Enigma_Key_Values[] =
    {0, 1, 2, 3, 4, 5, 6, 7, 255, 254, 253, 252, 251, 250, 249};
keySigType EnigmaKey[] =
    {C, SF, SFSC, SFSCSG, SFSCSGSD, SFSCSGSDSA,
    SFSCSGSDSASE, SFSCSGSDSASESB, FB, FBFE, FBFEFA, FBFEFAFD,
    FBFEFAFDG, FBFEFAFDGFC, FBFEFAFDGFCFF};
```

The information about time signature is stored as a combination of two numbers in an ETF file, the first specifying the number of beats in the time signature and the second the duration in EDUs of these beats, for example 3/4 is usually represented by “3 1024” and 6/8 “6 512”. CPNView stores time signature as a rational number and so the ETF version needed to be converted in a numerator and denominator. The number representing the numerator was readily available in the form needed for a CPNView numerator and was stored as such.

Two integer arrays were created to represent the time signature denominators supported by CPNView and Enigma files, with the matching denominators in the corresponding position, as shown below.

```
int EnigmaTSDenoms [ ] =
    { 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192};
int CPNTSDenoms [ ] =
    { 256, 128, 64, 32, 16, 8, 4, 2, 1, 0};
```

The number representing the denominator in the Enigma file was checked against those in the “EnigmaTSDenoms” array and if found the corresponding element in the “CPNTSDenoms” array was recorded as being the denominator of the piece of music.

This method was used throughout the program to translate a number found in an ETF file to a variable type that was recognizable to CPNView. The pseudo-code for this resembles :

```
Locate number in Enigma file
int count;
for(count = 0; count < Max number in array ; count++)
{
    if( Enigma Number == Number in Enigma Array [ count ] )
    {
        CPN Number = Number in CPN Array [ count ] ;
    }
}
```

A separate set of arrays had to be created to translate compound time signatures like 6/8 which could be represented as two dotted crotchet beats. The method used was as above, but here the numerator was multiplied by three to convert Enigma code like “4 1536” to a 12/8 time signature and “2 384” to 6/64.

CPNView allows for time signatures with the denominators 256 and 128, but Finale does not support these denominators. However I have allowed for the existence of these in the translation to allow for future expansion by Finale.

4.3.2 Clefs

The information regarding clefs was translated from Enigma file format to CPNView format in the same way as described above. The number representing the clef type in the Enigma file is checked against an array of possible numbers and if found the corresponding type

listed in the matching CPNView array list is stored as the clef of the piece.

```
int Enigma_Clef_Types [ ] = {0,1,2,3,4,5,6,7};  
clefType CPN_Clef_Types [ ] = {TREBLE, ALTO, TENOR, BASS, NOCLEF,  
NOCLEF, NOCLEF,BARITONE};
```

A number of the CPNView clef types have been listed as “NOCLEF” because CPNView does not support some of the clefs that Finale does. Finale allows the creation of two types of tenor clef- a G clef with an “8vb” marking and an F Clef, a percussion clef, and a contrabass clef (an octave lower than a bass clef).

4.3.3 Durations

The durations of a Finale file are represented in the entry record, which is part of the entry pool of an ETF file. As with pitch, the translation of duration was built up from a number of smaller tasks. The method of using arrays with matching elements in CPNView and Enigma types was used again here. Arrays were created, as shown below, with all the possible note duration values in both representation types.

```
int Enigma_Durations [ ] = { 8192, 4096, 2048, 1024, 512, 256, 128, 64, 32};  
durType CPN_Durations [ ] = { N0, N1, N2, N4, N8, N16, N32, N64, N128};
```

Thus if the number in the ETF file matches one of the Enigma duration array elements the corresponding array element in the array of CPNView duration types is stored as the duration for the CPNView score. The next step involved translating the durations of dotted notes.

Three arrays were created to represent the durations of notes with one, two and three dots as represented in the Enigma files. If the number extracted from the Enigma file matched a

number in one of these arrays, it was recorded that the note had one, two or three dots attached, depending on which array the number was found in. The duration of the note itself, without the dot was translated by comparing the element number at which the Enigma number was found with an array of CPNView duration type arranged in the same order.

```
int Enigma_Dotted_Notes1[] = { 48, 96, 192, 384, 768, 1536, 3072,
                               12288};
int Enigma_Dotted_Notes2[] = { 56, 112, 224, 448, 896, 1792, 3584, 7168,
                               14336};
int Enigma_Dotted_Notes3[] = { 60, 120, 240, 480, 960, 1920, 3840,
                               7680, 15360};
durType CPN_Dotted_Notes[] = { N128, N64, N32, N16, N8, N4, N2, N1, N0};
```

4.3.4 Barlines

In the “isMSLine” function the bitwise AND operator was used to determine if certain bits were set in the number that represented barlines. This was because the inclusion of certain symbols in a bar altered this number (see section 3.4), but the type of barline could still be determined by the bit structure of the number. Checking the bit structure of the numbers ensures that if Finale changes their representation of barlines in the Enigma code (some changes have been made regarding this particular line of code between version 3.0 and 3.5.1), the program will still be able to translate the code representing the type of barline.

The placement of the lines of code that represents the type of barline means that the information about every barline of the piece is grouped together in the file before any information about the actual notes (pitch and duration) is encountered. As each type of barline was translated from the number in the Enigma file it was entered into an array of barlines (of

CPNView barline type). The information in this array was not accessed again until the barlines were being output to a CPNView score. As the type of barline for each bar was stored in the array the placement of the barlines in the CPNView score was determined by recording the duration of notes and rests. Each duration was added to the last and when a full bar of the score had been output the barline for that particular bar was then added.

4.3.5 Anacrusis

Due to the method by which barlines are inserted into a CPNView score the code had to be altered slightly to allow for the possibility of there being an anacrusis in the opening. A function called “IsUpBeat” determines whether the first bar is a full bar or not and records the duration of the upbeat if this proves positive. In main the Enigma value of the anacrusis is converted to a CPNView rational duration type by the use of arrays as before. The elements of the arrays are shown below:

int Enigma_upbeats[]	Rat CPN_upbeats[]
32	Rat(1,128)
48	Rat(3,256)
56	Rat(7,512)
64	Rat(1,64)
96	Rat(3,128)
112	Rat(7,256)
128	Rat(1,32)
192	Rat(3,64)
224	Rat(7,128)
256	Rat(1,26)
384	Rat(3,32)
448	Rat(7,64)
512	Rat(1,8)
768	Rat(3,16)
896	Rat(7,32)
1024	Rat(1,4)
1536	Rat(3,8)
1792	Rat(7,16)
2048	Rat(1,2)

3072
3840

Rat(3,4)
Rat(15,16)

The rational duration of the notes of the anacrusis are taken into account in the section of code that inserts barlines into a score. The duration of the anacrusis is recorded and the durations of individual notes are added together to form a kind of compound duration value and when this value equals that of the anacrusis a barline is inserted into the score.

If an anacrusis is present in the Finale score the last bar of the music may be incomplete also and this had to be allowed for, and a closing barline inserted here if needed. The duration value of the final bar was worked out by subtracting the value of the anacrusis from the maximum value possible in a bar of the particular time signature.

4.3.6 Pitch

The translating of the Enigma file's TCD (tone centre displacement) was broken up into many steps. The first step undertaken to identify pitch was to treat all notes as diatonic degrees of the C major scale and to match the numbers representing these pitches to their note names and octaves. Numbers were identified throughout the program by using the CPNView StringIterator member functions "isNum", which tests to see if the next non-blank entity in a string is a number, and "num" which returns the value of a number. Positive pitch numbers could be recorded this way but negative numbers posed a slight problem and had to be dealt with separately. They were located by searching for the negative character "-" and then recording the number that immediately followed this. The arrays to represent the possible positive and negative TCD numbers were kept separate in the program because of the mirror like

representation around the tonic (a tone above the tonic = 16 and a tone below the tonic = -16).

These arrays were arranged in the order of a C major scale which corresponds to the arrangement of the array of CPNView note names. In this way when the TCD was located in a file the number representing pitch could be translated into the equivalent note name.

While studying various TCD values a pattern appeared for the representation of accidentals which was related to their bit structure. Sharps and double sharps were simply represented by the addition of 1 and 2 onto the TCD of a note. Flats and double flats are represented by -1 and -2 in the low order bit. It was determined by this that the first bit in the low order nybble was set to 0 to indicate a positive alteration and 1 to indicate a negative alteration on the note. Thus -1 (flat) would be represented by “ 1 0 0 1 “ in the low order nybble of the TCD and -2 (double flat) by “ 1 0 1 0” (decimal 9 and 10). This corresponds to the pattern discerned in TCDs, an example of accidentals on the tonic in C major is shown below:

Natural	Sharp	Double Sharp	Flat	Sharp
0	1	2	9	10

The next step was to get the program to identify the TCD in keys other than C major. By the time main calls this particular function the key signature of the piece has been found. The values of the TCD as translated in C major would be the same in any other key, the tonic of a key will always be zero and the octave above 112 etc. This meant that the TCD of notes in any other key could be worked out by displacing the translated values of C major by the relevant number of places. A list was made of the displacement values each key signature has from C

major, for example the TCD and note names of G major are the same as those of C major displaced four times. Thus if the key was G major all the TCD calculations for C major would be displaced by a value of 4. The arrays used to identify the value each key should be displaced by are shown below:

```
keySigType EnigmaKey [ ] =
    {C,    SF,  SFSC,  SFSCSG,  SFSCSGSD,  SFSCSGSDSA,
     SFSCSGSDSASE, SFSCSGSDSASESB,
     FB,   FBFE, FBFEFA, FBFEFAFD,  FBFEFAFDG,
     FBFEFAFDGFC, FBFEFAFDGFCFF};

int keyDisplacements [ ] =
    { 0,  4,  1,  5,  2,  6,  3,  0,
      3,  6,  2,  5,  1,  4,  0};
```

When this displacement value was known the note names could be worked out. Separate arrays were created that contained a list of the possible positive and negative TCD values. A single array was created to store the note names that these TCD would suggest and the arrays of TCD values are arranged in reverse order of each other so that the elements of the note names array match the elements of the TCD arrays.

```
char Pitch_Names [ ] =          {"CDEFGABCDEFGFGABCDEFGFGABC"};

int Pitch_Numbers_Positive [ ]= {0,  16,32, 48,  64, 80, 96, 112,128, 144, 160, 176,
192, 208, 224, 240, 256, 272, 288, 304, 320, 336};

int Pitch_Numbers_Negative [ ] = {336,320, 304, 288, 272, 256, 240, 224, 208, 192,
176, 160, 144, 128, 112, 96,  80, 64, 48, 32, 16};
```

The code for translating the positive and negative TCD values into the individual elements of a CPNView note are shown below.

```

if(TCD)                                // ( TCD is TRUE if the TCD is a positive number)
{
    degreePos = stri.num();
    degreePos = degreePos + (KeyDisplacement * 16);
}
for(i=0;i<Pos_Pitches;i++)
{
    if( degreePos == Pitch_Numbers_Positive[i])
    {
        Name = Pitch_Names [i];
        CPN_accid = CPN_Accid[0];
    }

    if(degreePos == (Pitch_Numbers_Positive[i] +1) )
    {
        Name = Pitch_Names [i];
        CPN_accid = CPN_Accid[1];
    }
    if(degreePos == (Pitch_Numbers_Positive[i]+2) )
    {
        Name = Pitch_Names [i];
        CPN_accid = CPN_Accid[2];
    }

    if(degreePos == (Pitch_Numbers_Positive[i]+9))
    {
        Name = Pitch_Names [i];
        CPN_accid = CPN_Accid[3];
    }
    if(degreePos == (Pitch_Numbers_Positive[i]+10))
    {
        Name = Pitch_Names [i];
        CPN_accid = CPN_Accid[4];
    }
}

if ( stri.nextChar ( '-' ) )           // ( if number is negative)
{
    degreeNeg = stri.num();
    if ( ks != C)
    {
        if ( ks == SFSC)
        {
            degreeNeg = degreeNeg - (KeyDisplacement*16);
        }

        else
        {
            KeyDisplacement = KeyDisplacement -1;
            degreeNeg = degreeNeg + (KeyDisplacement * 16);
        }
    }
}

```

```

    }
}

for( i=0; i<Neg_Pitches; i++ )
{
    if(degreeNeg == Pitch_Numbers_Negative[i] )
        Name = Pitch_Names [i];

        if(degreeNeg == (Pitch_Numbers_Negative[i] - 1) )
        {
            Name = Pitch_Names [i];
            CPN_accid = CPN_Accid[1];
        }
        if(degreeNeg == Pitch_Numbers_Negative[i] - 2 )
        {
            Name = Pitch_Names [i];
            CPN_accid = CPN_Accid[2];
        }
        if(degreeNeg == Pitch_Numbers_Negative[i] - 9 )
        {
            Name = Pitch_Names [i];
            CPN_accid = CPN_Accid[3];
        }
        if(degreeNeg == Pitch_Numbers_Negative[i] -10 )
        {
            Name = Pitch_Names [i];
            CPN_accid = CPN_Accid[4];
        }
    }
}

```

The octave of the TCD was translated by a set of if statements that set the range of an octave in TCD numbers:

```

if (degreeNeg >=224 && degreeNeg <336) octave = 2;
else if(degreeNeg >=112 && degreeNeg <224) octave = 3;
else if(degreeNeg >0 && degreeNeg <112) octave = 4;

else if(degreePos >=0 && degreePos <112) octave = 5;
else if(degreePos >=112 && degreePos <224) octave = 6;
else if(degreePos >=224 && degreePos <336) octave = 7;

```

This set of statements determined the octave range of notes for a C major scale. As the TCD values are relative to the key signature these statements would have caused the octave to change at each tonic (-112, 0, 112, 224, etc) and this needed to be compensated for by

forcing the octave value to change on the note C only so that C4 = -112, C5 = 0, C6 = 112, etc. Separate alterations to the octave value was devised for the positive and negative TCD values, as shown below.

```

if(ks != C)
{
    for(int disp = 1; disp<=KeyDisplacement ; disp ++)
    {
        if(degreeNeg %112 !=0)
        {
            for(int oct = 0;oct <448;oct+=112)
            {
                if(degreeNeg == ( 16 * disp) + oct )
                    octave = octave + 1;
            }
        }
    }

    for(int disp = KeyDisplacement; disp>=1; disp --)
    {
        for(int oct = 0; oct <448;oct +=112)
        {
            if(degreePos == (112 - (disp * 16) +oct) )
            {
                if(degreePos %112 !=0) octave = octave +1;
            }
        }
    }
}

```

Most accidentals that appear in music notation have been translated successfully.

However, if the note on which the accidental appears is already displaced by the key signature (flattened or sharpened), the accidental will be incorrectly translated. For example, if the seventh degree of the scale in G major appears as a natural in the notation, it is translated as a flat because the program recognises that the note has been altered down a semitone. In the future this may be corrected by using CPNViews member functions, such as “getKeySigAdjust” to

determine if a note is displaced as a consequence of the key signature of the music and if it is displaced, this would have to be taken into consideration when assigning the accidental.

4.3.7 Articulation

A function for the extraction of articulation information is included in the program. The code is a framework for future work on the translation of articulation elements. Due to the fact that the information about articulation is contained in a group, in a different place in the Enigma file than the note to which they are attached, difficulties were encountered in this area. The switch and case statement gives the equivalent CPNView types of the numerical values that represent articulation in an Enigma file. CPNView stores articulation as an enumerated type “attrType” (defined in “ATTRTYPE.H”). The corresponding values for each is shown below.

Enigma file value	CPNView attrtype
1	STACCATO
2	ACCENT
3	ACCENT,STACCATO
4	ACCENT
5	ACCENT,STACCATO
6	TENUTO
7	TENUTO,STACCATO
8	FERMATA
9	
10	LETTER_TRILL
11	
12	
13	UP_BOW
14	DOWN_BOW
15	TURN
16	PED
17	REL
18	
19	
20	GRACE_NOTE
21	HARMONIC
22	HARMONIC
23	PLUS
24	BREATH_MARK

25	TREMELO
26	TREMELO
27	TREMELO
28	
29	

Articulation types with no corresponding CPNView type are listed in section 3.6.

The problem of translating these articulation types might be approached by a method similar to that used in translating barlines types. The number of the entry to which the articulation refers to could be stored along with the CPNView type in a struct containing these two items. When the note to which the articulation is linked is then translated, the relevant articulation could be created with the note.

4.4 Creation of the CPNView Score

To create a CPNView score a variable of type Score had to be defined. In the program the variable was called “score” so that it would be very obvious that a CPNView score was being dealt with. The maximum number of staves was set to 1, as the translation process deals with monophonic music only at this stage. A “ScoreIterator” was then created for the score so that information could be extracted from the CPNView score that had been created. When the data from the Enigma file was translated, the member function of the class ScoreIterator were used to build a CPNView Score. Objects representing music were added using the “+” (plus) operator, which allow the user to append the score with objects, for example, if all the information about a note object was known a note could be added to the score by using the command `si + note()`. (the parameters for that particular note would be included in the brackets) .

Due to the structure of Enigma files the order of the objects needed to create a score are not all in the order desired and so variables set to TRUE and FALSE were used to ensure that the code that created the clef for the score was carried out after the code to create the key and time signatures.

Before the score is printed to screen the commands “resetScoreStart()”, “alignStart()” and “renumberBars()” are used to initialize the score and correct any duration displacement that might have been caused by the presence of anacrusis. The function call “alignStart()” was a private function of class Score in “Score.h” and it was made public so that it could be accessed from the program.

Chapter 5. Testing, Further Work and Conclusions.

5.1 Testing

The program was tested throughout the development of the translation code in order to ensure that each step was successfully working before subsequent code was added which would have made problems more difficult to find.

A separate group of test pieces of music were used when the program was completed. The pieces were chosen because each exhibited some unique characteristics that would be useful in testing the success of the translation from Enigma code to a CPNView score. The first piece of music used to test the operation of the program was “Oh Susannah”, a simple well known melody which Finale uses as a tutorial example. This piece was chosen because it is a relatively simple melody in C major. Fig.1 illustrates the notation of the this tune and Fig. 1a shows the CPNView score that is output when translated.

The next pieces of music that were used to test the operation of the program were Irish traditional music tunes. Both were chosen to test the programs capabilities in translating keys other than C major. The first is a polka entitled “Peg Ryan’s” and the second, “Gander in the Pratie Hole” is a jig. Fig. 2 and 2a illustrate the polka and Fig. 3 and 3a show the jig and it’s output.

An excerpt from C.P.E.Bach’s Concerto in G major for flute and piano was also used to test the translation process. This piece of music was chosen because it involves a large range of notes, rests and some accidentals. Fig x shows the excerpt used (Movement 1 bb.272 - 278) and Fig y shows the translation of the Enigma file representing this piece.

The accidental G# occurs in the Bach melody and the program successfully translates this. However in the the jig “Gander in the Pratie Hole” and the Bach fragment an accidental occurs which the program can not yet interpret. The key signature of the jig indicates that the note C is sharpened and when the note C natural is encountered in the piece the program to translate interprets the naturalised note as a flattened note. The F natural in the Bach melody is interpreted similarly and this feature of the translation process is one that will benefit from future work.

The testing phase shows that a CPNView score can be created from an Enigma file which successfully represents clef, time and key signatures, barlines, repeats, anacrusis, pitches, note names and octaves for diatonically based pieces, with only slight adjustment needed to successfully translate all accidentals possible.

5.2 Further Work

Proposals for further work are very much bound up in the achievements of the current project. Although not all musical elements of a notated score have been translated, most have been located in the Enigma file and the data concerning them documented. Due to time constraints on this project and the large body of research that needed to be carried out before the translation process began, the conversion of these elements could be carried out as an augmentation of the current location and conversion program. The system for determining the accidental on notes could be fully implemented as could the location and translation of features such as slurs and dynamics.

The main area for further work as regards this project lies in the translation of polyphonic music, which is a direct extension of the processes used to translate monophonic music.

Further work might also involve creating new entities in CPNView for those symbols of Finale notation that it does not support, for example, articulation and clef types.

5.3 Conclusion

The aim of this project has been to produce a functional software program to convert Finale's Enigma file format to a format readable by CPNView. The program will accept an Enigma file as input and locate and translate the elements of the file needed to build a musical score and output these in the form of a CPNView score. The implication of this is that CPNView will now be able to read and hence analyse and represent music that is stored on computer as Enigma files. A large number of systems currently exist for the encoding of music notation and rather than have to translate a musical score into one of these languages, this translator allows score notation files to be used directly with CPNView, without the need for transcription into ALMA, KERN, or similar musical codes.

APPENDIX 1: Program

```
#include "score.h"
#include <fstream.h>

using namespace CPN;

ifstream fin("polka.etf");          // ENTER THE NAME OF THE FILE TO BE TRANSLATED

const int No_of_Bars = 100;
const int No_Enigma_Keys = 15;
int Enigma_Key_Values[No_Enigma_Keys] =
    {0, 1, 2, 3, 4, 5, 6, 255, 254, 253, 252, 251, 250,
     249};
keySigType EnigmaKey[No_Enigma_Keys] =
    {C, SF, SFSC, SFSCSG, SFSCSGSD,
     SFSCSGSDSA, SFSCSGSDSASE,
     SFSCSGSDSASESB, FB, FBFE, FBFEFA,
     FBFEFAFD, FBFEFAFDG, FBFEFAFDGFC,
     FBFEFAFDGFCFF};
int keyDisplacements[No_Enigma_Keys] =
    { 0, 4, 1, 5, 2, 6, 3, 0, 3, 6, 2, 5, 1, 4, 0};

const int No_of_Names = 38;
char Pitch_Names[No_of_Names] = {"CDEFGABCDEFGABCDEFGABCDEFGABCDEFGAB"};
const int Pos_Pitches = 29;
int Pitch_Numbers_Positive[Pos_Pitches] = {0, 16, 32, 48, 64, 80, 96, 112, 128, 144, 160,
    176, 192, 208, 224, 240, 256, 272, 288, 304, 320,
    336, 352, 368, 384, 400, 416, 432, 448 };
const int Neg_Pitches = 28;
int Pitch_Numbers_Negative[Neg_Pitches] = {448, 432, 416, 400, 384, 368, 352, 336, 320, 304,
    288, 272, 256, 240, 224, 208, 192, 176, 160, 144,
    128, 112, 96, 80, 64, 48, 32, 16};

const int No_Enigma_Denoms = 10;
int EnigmaTSDenoms[No_Enigma_Denoms] = { 16, 32, 64, 128, 256, 512, 1024, 2048, 4096,
    8192};
int CPNTSDenoms[No_Enigma_Denoms] = { 256, 128, 64, 32, 16, 8, 4, 2, 1, 0};

const int No_of_Groups = 5;
int Enigma_Beat_Denoms[No_of_Groups] = { 192, 384, 768, 1536, 3072};
int CPNTSDenoms2[No_of_Groups] = { 64, 32, 16, 8, 4};

const int No_Dotted_Notes = 9;
int Enigma_Dotted_Notes1[No_Dotted_Notes] = { 48, 96, 192, 384, 768, 1536, 3072, 6144, 12288};
int Enigma_Dotted_Notes2[No_Dotted_Notes] = { 56, 112, 224, 448, 896, 1792, 3584, 7168,
    14336};
int Enigma_Dotted_Notes3[No_Dotted_Notes] = { 60, 120, 240, 480, 960, 1920, 3840, 7680,
    15360};
durType CPN_Dotted_Notes[No_Dotted_Notes] = { N128, N64, N32, N16, N8, N4, N2, N1, N0};

const No_Clef = 8;
```

```

int Enigma_Clef_Types [No_Clef] = {0, 1, 2, 3, 4, 5, 6, 7};
clefType CPN_Clef_Types [No_Clef] = { TREBLE, ALTO,TENOR, BASS, NOCLEF, NOCLEF,
                                      NOCLEF, BARITONE};

const No_of_Durations = 9;
int Enigma_Durations[No_of_Durations] = {8192, 4096, 2048,1024, 512, 256, 128, 64, 32};
durType CPN_Durations[No_of_Durations] = { N0, N1, N2, N4, N8, N16, N32, N64, N128};
const AccidNo = 6;
int Enigma_Accid [AccidNo] = {0, 1, 2, 9,10};
accidType CPN_Accid [AccidNo] = {NOACCID, S, DS, F, DF, N};

const int No_upbeats = 21;

int Enigma_upbeats[No_upbeats] = { 32, 48, 56, 64, 96, 112, 128, 192, 224, 256, 384, 448,
                                   512, 768, 896, 1024, 1536, 1792, 2048, 3072, 3840};

Rat CPN_upbeats[No_upbeats] = { Rat(1,128), Rat(3,256), Rat(7,512), Rat(1,64), Rat(3,128),
                                Rat(7,256), Rat(1,32), Rat(3,64), Rat(7,128), Rat(1,26),
                                Rat(3,32),Rat(7,64), Rat(1,8), Rat(3,16), Rat(7,32), Rat(1,4),
                                Rat(3,8), Rat(7,16), Rat(1,2), Rat(3,4), Rat(15,16) };

//function prototypes
int getLine(String & str);

int isVersion(String & str, String & versionNo, int & yearNo);

int isVersionNumber(StringIterator & stri, String & versionNumber);

int isMSLine(String & str, keySigType & ks, int & numerator, int & denominator, int & barNo,
             barType & BarLine,attrType & RepeatType);

int isClef(String & str, int & StaveNo,int & BarNo_Clef, clefType & Name_ClefType);

int Entry_Record(String & str,int & No_of_Entry,int & PrevLink,int & NextLink,durType & duration,
                int & No_of_Dots,int & isANote,int & ARTIC);

int Note_Record (String & str,char & Name,int & octave,keySigType & ks,int & KeyDisplacement,
                accidType & CPN_accid);

int IsUpBeat(String & str,int & upbeat);

int getArtic(String & str);

//***** MAIN *****//
int main()
{
    String str;

    Score score;
    score.setMaxStaves(1);

    ScoreIterator si(score);

```

```

int initialsDone = FALSE;
int noteRestMode = FALSE;
int tsNumerator = 0, tsDenominator = 0;
int barNo = 0, No_of_Dots = FALSE, graceNote = FALSE;
int i = 0, GR = 0, upbeat = 0;
barType BarLines[No_of_Bars];

int ARTIC = FALSE, ARTIC_ADDED = FALSE;

while( getLine(str))
{
    String versionNo;
    int yearNo = 0;
    keySigType ks;

    clefType Name_ClefType;
    int StaveNo = 0, BarNo_Clef = 0;
    int No_of_Entry = 0, PrevLink = 0, NextLink = 0;
    int TCD = 0, isANote=0;
    durType duration;
    barType BarLine;
    attrType RepeatType;
    char Name;
    int octave=0, KeyDisplacement = 0;
    accidType CPN_accid;

    if ( isVersion(str, versionNo, yearNo))
    {
        cout << "\nVersion entry recognised.";
        cout << "\nVersion is " << versionNo << ", year is " << yearNo<<"\n";
    }
    else if ( isMSLine(str, ks, tsNumerator, tsDenominator, barNo, BarLine,
        RepeatType))
    {
        BarLines[i] = BarLine;
        i++;
    }
    else if ( isClef(str, StaveNo, BarNo_Clef, Name_ClefType))
    {
        if ( !initialsDone && StaveNo ==1)
        {
            initialsDone = TRUE;
            si + Clef(Name_ClefType);
            si + KeySig(ks);
            si + TimeSig(tsNumerator, tsDenominator);
        }
    }
}

```

```

    }
    else

        if(Entry_Record(str, No_of_Entry, PrevLink, NextLink, duration, No_of_Dots,
            IsANote, ARTIC))

        {

            if ( isANote)
            {

                noteRestMode = TRUE;

            }

            else
            {

                si + Rest(duration,No_of_Dots);
                No_of_Dots = FALSE;

            }

        }
    else
        if( noteRestMode && Note_Record ( str, Name, octave, ks, KeyDisplacement,
            CPN_accid))

        {

            if(! ARTIC)
            {

                si + Note ( Name, octave, CPN_accid,duration,No_of_Dots);
                No_of_Dots = FALSE;

            }

        }
    else if(IsUpBeat(str,upbeat))
    {

    }

}

si.locate();

i = 0;
Rat d(0,1);
Rat up(0,1);
int x = 0;

for(x = 0; x< No_upbeats; x++)
{

```

```

        if(upbeat == Enigma_upbeats[x]) up = CPN_upbeats[x];
    }

    while( si.step())
    {

        if(si.isA(NOTE) || si.isA(REST))
        {

            d += si.getRDuration();

            if(d == up)
            {
                si + Barline(BarLines[i]);
                i++;

                up = Rat(0,1);
                d = Rat(0,1);

            }

            if(si.hasAttribute(GRACE_NOTE))
            {

                d = (d -= si.getRDuration());

            }

            if(d == si.getTimeSig())
            {
                si + Barline(BarLines[i]);
                i++;

                d = Rat(0,1);

            }

        }

    }

    Rat rem(0,1);          // if last bar is not full bar
    rem = si.getTimeSig() - up ;

    if(rem == (0,1) ) si+ Barline(BarLines[i]);


    score.resetScoreStart();          // Resets initial bar information
    score.allignStart();              //
    score.renumberBars(0);            // Renumbers bars

```



```

        si.locate();
        do
        {
            cout << si<< "\n";

        } while ( si.step());

    } // end of main

//***** Function to read in file line by line *****/

int getLine(String & str)
{
    str = String();
    int more = TRUE;
    while ( more )
    {
        char c = fin.get();
        if ( fin.fail() )
        {
            if (str.length() == 0 ) return FALSE;
            else return TRUE;
        }

        if ( c >= 32 && c < 127 ) str += c;
        else if ( c == '\n' ) more = FALSE;
    }

    return TRUE;
}

//***** Function to get year no *****/

int isVersion(String & str, String & versionNo, int & yearNo)
{
    StringIterator tstri(str);

    versionNo = String();
    yearNo = 0;

    if ( tstri.heads("^01 \"Finale(R) ") )
    {
        if ( isVersionNumber( tstri, versionNo ) )
        {
            if ( tstri.heads("Copyright (c) ") )
            {

```

```

        if ( tstri.isNum())
        {
            yearNo = tstri.num();
            tstri.next();
            if ( tstri.heads("Coda Music Technology\\"))
            {
                return TRUE;
            }
        }
    }
}

return FALSE;
}

```

//***** Function to get version no *****//

```

int isVersionNumber(StringIterator & stri, String & versionNumber)
{

```

```

    StringIterator tstri = stri;
    if ( tstri.isNum())
    {
        versionNumber += tstri.get();
        ++tstri;
        if ( tstri.heads('.'))
        {
            versionNumber += '.';
            if ( tstri.isNum())
            {
                versionNumber += tstri.get();
                ++tstri;
                tstri.next();
                stri = tstri;
                return TRUE;
            }
        }
    }
    return FALSE;
}

```

//***** Function to get MS Line - key,time sig and type of barline *****//

```

int isMSLine(String & str, keySigType & ks, int & numerator, int & denominator, int & barNo,
    barType & BarLine,attrType & RepeatType)
{

```

```

    StringIterator tstri(str);
    int FirstNo=0,SecNo=0,TrdNo=0,FrNo=0,FifNo=0,LastNo=0;

    if(tstri.heads("^MS("))
    {
        if ( !tstri.isNum()) return FALSE;
    }

```

```

barNo = tstri.num();

if ( !tstri.nextChar('')) return FALSE;
tstri.next();

if ( !tstri.isNum()) return FALSE;
FirstNo = tstri.num();
tstri.next();

if (!tstri.isNum()) return FALSE;
SecNo = tstri.num();


int i = 0;
for(i = 0;i <No_Enigma_Keys;i++)
{
    if(SecNo == Enigma_Key_Values[i])
        ks = EnigmaKey[i];

}

tstri.next();

if( ! tstri.isNum()) return FALSE;
numerator = tstri.num();
tstri.next();

if ( !tstri.isNum()) return FALSE;
FrNo = tstri.num();


denominator = 0;
for (int count = 0; count < No_Enigma_Denoms; count++)
{
    if ( FrNo == EnigmaTSDenoms[count] )
        denominator = CPNTSDenoms[count];

}

for(int count = 0 ; count <No_of_Groups; count ++)
{
    if( FrNo == Enigma_Beat_Denoms[count])
    {
        denominator = CPNTSDenoms2[count];
        numerator = numerator*3;
    }

}
tstri.next();

if ( ! tstri.isNum()) return FALSE;

```

```

        FifNo = tstri.num();
        tstri.next();

        if ( !tstri.isNum()) return FALSE;
        LastNo = tstri.num();

        if ( (LastNo & 0x00000010) !=0) BarLine = L;    // normal
        if ( ((LastNo & 0x00000020) !=0) && ((LastNo & 0x00002000) !=0) )
            BarLine = LL; // double
        if ( (LastNo & 0x00000040) !=0) BarLine = H;    // solid
        if ( ((LastNo & 0x00002000) !=0) && ((LastNo & 0x00000040) !=0) &&
            ((LastNo & 0x00000010) !=0) ) BarLine = H;
        if ( ((LastNo & 0x00000020) !=0) && (LastNo & 0x00000010) !=0)
            BarLine = DOTTED;
        if( ((LastNo & 0x00002000) !=0) && ((LastNo & 0x00000FFF) == 0) )
            BarLine = INVISIBLE;

        if ( (LastNo & 0x00000004) != 0) BarLine = CLH;
        if ( (LastNo & 0x00000008) != 0) BarLine = HLC;
        if ( (LastNo & 0x00000006) != 0 || (LastNo & 00000007) != 0)
            RepeatType = REPEAT1;
        if ( (LastNo & 0x00000002) != 0) RepeatType = REPEAT2;
        if ( (LastNo & 0x00000001) != 0) cout << "\tTEXT REPEAT\t"; // Text repeat

        return TRUE;
    }

    return FALSE;
}

//***** Function to get clef type *****/

int isClef(String & str, int & StaveNo, int & BarNo_Clef, clefType & Name_ClefType)
{

    int EntryNo, ClefType;
    StringIterator stri(str);

    if(stri.heads("^GF("))
    {

        if(!stri.isNum()) return FALSE;
        StaveNo = stri.num();

        if(!stri.nextChar(',') ) return FALSE;
        if(!stri.isNum()) return FALSE;
    }

```

```

        BarNo_Clef = stri.num();

        if(!stri.nextChar('')) return FALSE;
        stri.next();

        if(!stri.isNum()) return FALSE;
        EntryNo = stri.num();
        stri.next();

        if(!stri.isNum()) return FALSE;
        ClefType = stri.num();

        for(int i=0;i<No_Clef;i++)
        {
            if(ClefType == Enigma_Clef_Types [i])
            {
                Name_ClefType = CPN_Clef_Types [i];

            }
        }

        return TRUE;

    }

    return FALSE;
}

//***** Function to get entry record *****/

int Entry_Record (String & str,int & No_of_Entry,int & PrevLink,int & NextLink,durType & duration,
                  int & No_of_Dots,int & isANote,int & ARTIC)
{
    StringIterator stri(str);
    int Pos,ExtEntry,NoNotes,Dur;
    int Byte1,Byte2,Byte3,Byte4,Byte5,Byte6,Byte7,Byte8;

    if( stri.heads("^eE("))
    {
        if(!stri.isNum()) return FALSE;
        No_of_Entry = stri.num();

        if(!stri.nextChar('')) return FALSE;
        stri.next();

        if(!stri.isNum()) return FALSE;
        PrevLink = stri.num();
        stri.next();

        if(!stri.isNum()) return FALSE;
        NextLink = stri.num();
        stri.next();
    }

```

```

if(!stri.isNum()) return FALSE;
Dur = stri.num();

for(int i= 0; i< No_of_Durations; i++)
{
    if(Dur == Enigma_Durations[i] ) duration = CPN_Durations[i];
}

for(int i = 0;i<No_Dotted_Notes;i++)
{
    if(Dur == Enigma_Dotted_Notes1[i])
    {
        duration = CPN_Dotted_Notes[i];
        No_of_Dots = 1;
    }
    else if(Dur == Enigma_Dotted_Notes2[i])
    {
        duration = CPN_Dotted_Notes[i];
        No_of_Dots = 2;
    }
    else if(Dur == Enigma_Dotted_Notes3[i])
    {
        duration = CPN_Dotted_Notes[i];
        No_of_Dots = 3;
    }
}

stri.next();

if(!stri.isNum()) return FALSE;
Pos = stri.num();
stri.next();

// ENTRY FLAG
if(!stri.nextChar('$')) return FALSE;
    // The next section stores each byte as a character because
    // there are no spaces separating the elements of the entry flag
    // and each needs to be located and stored individually.

Byte1 = stri.get();
++stri;
if(!stri.isNum()) return FALSE;
Byte2 = stri.get();
++stri;
if(!stri.isNum()) return FALSE;
Byte3 = stri.get();
if(Byte3 == 56)          ARTIC = TRUE;
    else if(Byte3 == 50)  ARTIC = TRUE;
++stri;
if(!stri.isNum()) return FALSE;
Byte4 = stri.get();

```

```

        ++stri;
        if(!stri.isNum()) return FALSE;
        Byte5 = stri.get();
        ++stri;
        if(!stri.isNum()) return FALSE;
        Byte6 = stri.get();
        ++stri;
        if(!stri.isNum()) return FALSE;
        Byte7 = stri.get();
        ++stri;
        if(!stri.isNum()) return FALSE;
        Byte8 = stri.get();
        ++stri;

        stri.next();

        if(!stri.isNum()) return FALSE;
        ExtEntry = stri.num();
        stri.next();
        if(ExtEntry == 8)          cout << "\tEntry is part of a groupette";
        if(ExtEntry == 4)          cout << "\tEntry is tied over from previous bar";

        if(!stri.isNum()) return FALSE;
        NoNotes = stri.num();
        if ( NoNotes ==0) isANote = FALSE;
        if ( NoNotes > 0) isANote = TRUE;

        return TRUE;
    }

return FALSE;
}

//***** Function to get note record information *****//

int Note_Record (String & str, char & Name,int & octave,keySigType & ks,int & KeyDisplacement,
                accidType & CPN_accid)
{
    int B1,B2,B3,B4,B5,B6,B7,B8;
    int i=0, TCD=0;
    int degreePos = 0,degreeNeg = 0;

    StringIterator stri(str);

    int j = 0;
    for(j=0;j<No_Enigma_Keys;j++)
    {
        if(ks == EnigmaKey[j])
        {
            KeyDisplacement= keyDisplacements[j];

```

```

    }
}
if(stri.heads(" ") || stri.heads(" ") )
{

    if(stri.isNum()) TCD = TRUE;

    if(TCD) degreePos = stri.num();      // POSITIVE TCD VALUE

    for(i=0;i<Pos_Pitches;i++)
    {
        if( degreePos == Pitch_Numbers_Positive[i])
        {
            Name = Pitch_Names[i+KeyDisplacement];
            CPN_accid = CPN_Accid[0];
        }

        if(degreePos == (Pitch_Numbers_Positive[i] +1) )
        {
            Name = Pitch_Names[i + KeyDisplacement];
            CPN_accid = CPN_Accid[1];
        }
        if(degreePos == (Pitch_Numbers_Positive[i]+2) )
        {
            Name = Pitch_Names[i + KeyDisplacement];
            CPN_accid = CPN_Accid[2];
        }

        if(degreePos == (Pitch_Numbers_Positive[i]+9))
        {
            Name = Pitch_Names[i + KeyDisplacement];
            CPN_accid = CPN_Accid[3];
        }
        if(degreePos == (Pitch_Numbers_Positive[i]+10))
        {
            Name = Pitch_Names[i+ KeyDisplacement];
            CPN_accid = CPN_Accid[4];
        }

    }

}

if(stri.nextChar('-'))      // NEGATIVE TCD VALUE
{
    degreeNeg = stri.num();

    for(i=0;i<Neg_Pitches;i++)
    {
        if(degreeNeg == Pitch_Numbers_Negative[i] )

```



```

        Name = Pitch_Names[i+KeyDisplacement]; //no accidental

        if(degreeNeg == (Pitch_Numbers_Negative[i] - 1) )           // sharp
        {
            Name = Pitch_Names[i+ KeyDisplacement];
            CPN_accid = CPN_Accid[1];
        }
        if(degreeNeg == Pitch_Numbers_Negative[i] - 2 )           //double sharp
        {
            Name = Pitch_Names[i+ KeyDisplacement];
            CPN_accid = CPN_Accid[2];
        }
        if(degreeNeg == Pitch_Numbers_Negative[i] - 9 )           //flat
        {
            Name = Pitch_Names[i+ KeyDisplacement];
            CPN_accid = CPN_Accid[3];
        }
        if(degreeNeg == Pitch_Numbers_Negative[i] -10 )           //double flat
        {
            Name = Pitch_Names[i+ KeyDisplacement];
            CPN_accid = CPN_Accid[4];
        }
    }
}

```

```

if( degreeNeg >=16 && degreeNeg <=112) octave = 4 ;
else if(degreeNeg >112 && degreeNeg <=224) octave = 3;
else if(degreeNeg >224 && degreeNeg <=336) octave = 2;
else if(degreeNeg >336 && degreeNeg <=448) octave = 1;
else if(degreeNeg >448) cout << "OCTAVE NOT RECOGNISED";

```

```

else if(degreePos >=0 && degreePos <112) octave = 5;
else if(degreePos >=112 && degreePos <224) octave = 6;
else if(degreePos >=224 && degreePos <336) octave = 7;
else if(degreePos >=336 && degreePos <448) octave = 8;
else if(degreePos == 448) octave = 9;
else if(degreePos >448) cout << "OCTAVE NOT RECOGNISED";

```

```

if(ks != C)
{

```

```

    for(int disp = 1; disp<=KeyDisplacement ; disp ++ )
    {
        if(degreeNeg %112 !=0)
        {
            for(int oct = 0;oct <448;oct+=112)
            {
                if(degreeNeg == ( (16 * disp) + oct) )
                    octave = octave + 1;
            }
        }
    }
}

```

```

for(int disp = KeyDisplacement; disp>=1; disp --)
{
    // to allow for each octave of the key
    for(int oct = 0; oct <448;oct +=112)
    {
        if(degreePos == (112 - (disp * 16) +oct) )
        {
            if(degreePos %112 !=0) octave = octave +1;

        }

    }

}

}

stri.next();

if(!stri.nextChar('$')) return FALSE; // NOTE FLAG

B1 = stri.get();
++stri;

if(!stri.isNum()) return FALSE;
B2 = stri.get();
++stri;

if(!stri.isNum()) return FALSE;
B3 = stri.get();
++stri;

if(!stri.isNum()) return FALSE;
B4 = stri.get();
++stri;

if(!stri.isNum()) return FALSE;
B5 = stri.get();
++stri;

if(!stri.isNum()) return FALSE;
B6 = stri.get();
++stri;

if(!stri.isNum()) return FALSE;
B7 = stri.get();
++stri;

```

```

        if(!stri.isNum()) return FALSE;
        B8 = stri.get();

        return TRUE;
    }
    return FALSE;
}
//***** Function to check if there is an upbeat *****//
int IsUpBeat(String & str,int & upbeat)
{
    StringIterator stri(str);
    if(stri.heads("^17(65534)") )
    {
        ++stri;
        if( !stri.isNum() ) return FALSE;
        stri.num();
        ++stri;
        if( !stri.isNum() ) return FALSE;
        stri.num();
        ++stri;
        if( !stri.isNum() ) return FALSE;
        stri.num();
        ++stri;
        if(!stri.nextChar('-') ) return FALSE;
        stri.num();
        ++stri;
        if( !stri.isNum() ) return FALSE;
        stri.num();
        ++stri;
        if( !stri.isNum() ) return FALSE;

        int isupbeat = stri.num();
        if(isupbeat >0)
        {
            upbeat = isupbeat;
            return TRUE;
        }
    }
    return 0;
}
//***** Function to find articulation symbol *****//

int getArtic(String & str)
{
    int artic = 0;
    StringIterator stri(str);
    if(stri.heads("^IM("))
    {

```

```

if(!stri.isNum()) return FALSE;
stri.num();
if(!stri.heads(',') ) return FALSE;

if(!stri.isNum()) return FALSE;
stri.num();

if(!stri.heads('') ) return FALSE;
stri.next();

if(!stri.isNum()) return FALSE;
artic = stri.num();

switch(artic)
{
    case 0: NO_ATTR;break;
    case 1: STACCATO; break;
    case 2: ACCENT; break;
    case 3: ACCENT; break;
    case 4: HEAVY_ACCENT;break;
    case 5: HEAVY_ACCENT; break;
    case 6: TENUTO;break;
    case 7: TENUTO;break;
    case 8: FERMATA;break;
    case 9: break; //mordent
    case 10: LETTER_TR;break;
    case 11: break; //mordent
    case 12: break;
    case 13: UP_BOW;break;
    case 14: DOWN_BOW;break;
    case 15: break; //turn
    case 16: PED;break;
    case 17: REL;break;
    case 18: break; //arpeggiated chord
    case 19: break ; //railroad tracks
    case 20: GRACE_NOTE;cout << "should be grace note";break;
    case 21: HARMONIC;break;
    case 22: HARMONIC;break;
    case 23: PLUS;break;
    case 24: BREATH_MARK;break;
    case 25: TREMOLO;break;
    case 26: break;
    case 27: break;
    case 28: break; // round brackets
    case 29: break;

}

return TRUE;
}

return FALSE;
}

```


APPENDIX 2: Test Tunes

Fig. 1

Oh Susannah



CPNView Score : “Oh Susannah”

Fig. 1a

```

* 0:Clef:TREBLE * Context:TREBLE
* 0:Keysig:ks: * Context:ks:
* 0:TimeSig:(2,2) * Context:(2,2)
* HV:0:** Rest:[2] {} cp=(0/1) cd=(1/2), rem=(1/2)
* HV:0:Note:C5 [3](1){} cp=(1/2) cd=(3/8), rem=(3/8)
* HV:0:Note:D5 [4] {} cp=(7/8) cd=(1/8), rem=(1/8)
* 0:Barline:Bar:/ 2 * Context:Bar:/ 2
* HV:0:Note:E5 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
* HV:0:Note:G5 [3] {} cp=(1/4) cd=(1/4), rem=(1/4)
* HV:0:Note:G5 [3](1){} cp=(1/2) cd=(3/8), rem=(3/8)
* HV:0:Note:A5 [4] {} cp=(7/8) cd=(1/8), rem=(1/8)
* 0:Barline:Bar:/ 3 * Context:Bar:/ 3
* HV:0:Note:G5 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
* HV:0:Note:E5 [3] {} cp=(1/4) cd=(1/4), rem=(1/4)
* HV:0:Note:C5 [3](1){} cp=(1/2) cd=(3/8), rem=(3/8)
* HV:0:Note:D5 [4] {} cp=(7/8) cd=(1/8), rem=(1/8)
* 0:Barline:Bar:/ 4 * Context:Bar:/ 4
* HV:0:Note:E5 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
* HV:0:Note:E5 [3] {} cp=(1/4) cd=(1/4), rem=(1/4)
* HV:0:Note:D5 [3] {} cp=(1/2) cd=(1/4), rem=(1/4)
* HV:0:Note:C5 [3] {} cp=(3/4) cd=(1/4), rem=(1/4)
* 0:Barline:Bar:/ 5 * Context:Bar:/ 5
* HV:0:Note:D5 [2] {} cp=(0/1) cd=(1/2), rem=(1/2)
* HV:0:** Rest:[3] {} cp=(1/2) cd=(1/4), rem=(1/4)
* HV:0:Note:C5 [4] {} cp=(3/4) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(7/8) cd=(1/8), rem=(1/8)
* 0:Barline:Bar:/ 6 * Context:Bar:/ 6
* HV:0:Note:E5 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
* HV:0:Note:G5 [3] {} cp=(1/4) cd=(1/4), rem=(1/4)
* HV:0:Note:G5 [3](1){} cp=(1/2) cd=(3/8), rem=(3/8)
* HV:0:Note:A5 [4] {} cp=(7/8) cd=(1/8), rem=(1/8)
* 0:Barline:Bar:/ 7 * Context:Bar:/ 7
* HV:0:Note:G5 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
* HV:0:Note:E5 [3] {} cp=(1/4) cd=(1/4), rem=(1/4)
* HV:0:Note:C5 [3](1){} cp=(1/2) cd=(3/8), rem=(3/8)
* HV:0:Note:D5 [4] {} cp=(7/8) cd=(1/8), rem=(1/8)
* 0:Barline:Bar:/ 8 * Context:Bar:/ 8
* HV:0:Note:E5 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
* HV:0:Note:E5 [3] {} cp=(1/4) cd=(1/4), rem=(1/4)

```

* HV:0:Note:D5 [3] {} cp=(1/2) cd=(1/4), rem=(1/4)
 * HV:0:Note:D5 [3] {} cp=(3/4) cd=(1/4), rem=(1/4)
 * 0:Barline:Bar:/ 9 * Context:Bar:/ 9
 * HV:0:Note:C5 [2] {} cp=(0/1) cd=(1/2), rem=(1/2)
 * HV:0:** Rest:[2] {} cp=(1/2) cd=(1/2), rem=(1/2)
 * 0:Barline:Bar:/ 10 * Context:Bar:/ 10
 * HV:0:Note:F5 [2] {} cp=(0/1) cd=(1/2), rem=(1/2)
 * HV:0:Note:F5 [2] {} cp=(1/2) cd=(1/2), rem=(1/2)
 * 0:Barline:Bar:/ 11 * Context:Bar:/ 11
 * HV:0:Note:A5 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
 * HV:0:Note:A5 [2] {} cp=(1/4) cd=(1/2), rem=(1/2)
 * HV:0:Note:A5 [3] {} cp=(3/4) cd=(1/4), rem=(1/4)
 * 0:Barline:Bar:/ 12 * Context:Bar:/ 12
 * HV:0:Note:G5 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
 * HV:0:Note:G5 [3] {} cp=(1/4) cd=(1/4), rem=(1/4)
 * HV:0:Note:E5 [3] {} cp=(1/2) cd=(1/4), rem=(1/4)
 * HV:0:Note:C5 [3] {} cp=(3/4) cd=(1/4), rem=(1/4)
 * 0:Barline:Bar:/ 13 * Context:Bar:/ 13
 * HV:0:Note:D5 [2](1){} cp=(0/1) cd=(3/4), rem=(3/4)
 * HV:0:Note:C5 [4] {} cp=(3/4) cd=(1/8), rem=(1/8)
 * HV:0:Note:D5 [4] {} cp=(7/8) cd=(1/8), rem=(1/8)
 * 0:Barline:Bar:/ 14 * Context:Bar:/ 14
 * HV:0:Note:E5 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
 * HV:0:Note:G5 [3] {} cp=(1/4) cd=(1/4), rem=(1/4)
 * HV:0:Note:G5 [3](1){} cp=(1/2) cd=(3/8), rem=(3/8)
 * HV:0:Note:A5 [4] {} cp=(7/8) cd=(1/8), rem=(1/8)
 * 0:Barline:Bar:/ 15 * Context:Bar:/ 15
 * HV:0:Note:G5 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
 * HV:0:Note:E5 [3] {} cp=(1/4) cd=(1/4), rem=(1/4)
 * HV:0:Note:C5 [3](1){} cp=(1/2) cd=(3/8), rem=(3/8)
 * HV:0:Note:D5 [4] {} cp=(7/8) cd=(1/8), rem=(1/8)
 * 0:Barline:Bar:/ 16 * Context:Bar:/ 16
 * HV:0:Note:E5 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
 * HV:0:Note:E5 [3] {} cp=(1/4) cd=(1/4), rem=(1/4)
 * HV:0:Note:D5 [3] {} cp=(1/2) cd=(1/4), rem=(1/4)
 * HV:0:Note:D5 [3] {} cp=(3/4) cd=(1/4), rem=(1/4)
 * 0:Barline:Bar:/ 17 * Context:Bar:/ 17
 * HV:0:Note:C5 [2] {} cp=(0/1) cd=(1/2), rem=(1/2)
 * HV:0:** Rest:[2] {} cp=(1/2) cd=(1/2), rem=(1/2)
 * 0:Barline:Bar// 18 * Context:Bar// 18

Fig. 2
Polka

Peg Ryan's



CPNView Score : Peg Ryan's

Fig. 2a

```

* 0:Clef:TREBLE * Context:TREBLE
* 0:Keysig:ks:#F * Context:ks:#F
* 0:TimeSig:(2,4) * Context:(2,4)
* HV:0:Note:B5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
* HV:0:Note:E5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
* 0:Barline:Bar:/ 2 * Context:Bar:/ 2
* HV:0:Note:B5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
* HV:0:Note:E5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
* 0:Barline:Bar:/ 3 * Context:Bar:/ 3
* HV:0:Note:G5 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
* HV:0:Note:A5 [4](1){} cp=(1/4) cd=(3/16), rem=(3/16)
* HV:0:Note:B5 [5] {} cp=(7/16) cd=(1/16), rem=(1/16)
* 0:Barline:Bar:/ 4 * Context:Bar:/ 4
* HV:0:Note:A5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
* HV:0:Note:G5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
* HV:0:Note:E5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
* 0:Barline:Bar:/ 5 * Context:Bar:/ 5
* HV:0:Note:B5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
* HV:0:Note:E5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
* 0:Barline:Bar:/ 6 * Context:Bar:/ 6
* HV:0:Note:B5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
* HV:0:Note:E5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
* 0:Barline:Bar:/ 7 * Context:Bar:/ 7
* HV:0:Note:G5 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
* HV:0:Note:A5 [4](1){} cp=(1/4) cd=(3/16), rem=(3/16)
* HV:0:Note:B5 [5] {} cp=(7/16) cd=(1/16), rem=(1/16)
* 0:Barline:Bar:/ 8 * Context:Bar:/ 8
* HV:0:Note:A5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
* HV:0:Note:G5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
* HV:0:Note:G5 [3] {} cp=(1/4) cd=(1/4), rem=(1/4)
* 0:Barline:Bar::/I 9 * Context:Bar::/I 9
* HV:0:Note:B5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)

```

* HV:0:Note:D6 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:B5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
 * HV:0:Note:A5 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
 * 0:Barline:Bar:/ 10 * Context:Bar:/ 10
 * HV:0:Note:A5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
 * HV:0:Note:G5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:E5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
 * HV:0:Note:D5 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
 * 0:Barline:Bar:/ 11 * Context:Bar:/ 11
 * HV:0:Note:G5 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
 * HV:0:Note:A5 [4](1){} cp=(1/4) cd=(3/16), rem=(3/16)
 * HV:0:Note:B5 [5] {} cp=(7/16) cd=(1/16), rem=(1/16)
 * 0:Barline:Bar:/ 12 * Context:Bar:/ 12
 * HV:0:Note:A5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
 * HV:0:Note:G5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:E5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
 * HV:0:Note:D5 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
 * 0:Barline:Bar:/ 13 * Context:Bar:/ 13
 * HV:0:Note:B5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
 * HV:0:Note:D6 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:B5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
 * HV:0:Note:A5 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
 * 0:Barline:Bar:/ 14 * Context:Bar:/ 14
 * HV:0:Note:A5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
 * HV:0:Note:G5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:E5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
 * HV:0:Note:D5 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
 * 0:Barline:Bar:/ 15 * Context:Bar:/ 15
 * HV:0:Note:G5 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
 * HV:0:Note:A5 [4](1){} cp=(1/4) cd=(3/16), rem=(3/16)
 * HV:0:Note:B5 [5] {} cp=(7/16) cd=(1/16), rem=(1/16)
 * 0:Barline:Bar:/ 16 * Context:Bar:/ 16
 * HV:0:Note:A5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
 * HV:0:Note:G5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:G5 [3] {} cp=(1/4) cd=(1/4), rem=(1/4)
 * 0:Barline:Bar:/I 17 * Context:Bar:/I 17]

Fig. 3

Gander in the Pratie Hole



CPNView Score: “Gander in the Pratie Hole”

Fig. 3a

```

* 0:Clef:TREBLE * Context:TREBLE
* 0:Keysig:ks:#F#C * Context:ks:#F#C
* 0:TimeSig:(6,8) * Context:(6,8)
* HV:0:Note:F5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
* HV:0:Note:A5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
* HV:0:Note:F5 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
* HV:0:Note:A5 [4] {} cp=(1/2) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(5/8) cd=(1/8), rem=(1/8)
* 0:Barline:Bar:/ 2 * Context:Bar:/ 2
* HV:0:Note:G5 [3](1){} cp=(0/1) cd=(3/8), rem=(3/8)
* HV:0:Note:E5 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
* HV:0:Note:F5 [4] {} cp=(1/2) cd=(1/8), rem=(1/8)
* HV:0:Note:G5 [4] {} cp=(5/8) cd=(1/8), rem=(1/8)
* 0:Barline:Bar:/ 3 * Context:Bar:/ 3
* HV:0:Note:F5 [5] {} cp=(0/1) cd=(1/16), rem=(1/16)
* HV:0:Note:G5 [5] {} cp=(1/16) cd=(1/16), rem=(1/16)
* HV:0:Note:A5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
* HV:0:Note:F5 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
* HV:0:Note:A5 [4] {} cp=(1/2) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(5/8) cd=(1/8), rem=(1/8)
* 0:Barline:Bar:/ 4 * Context:Bar:/ 4
* HV:0:Note:G5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
* HV:0:Note:E5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
* HV:0:Note:C5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
* HV:0:Note:A5 [4] {} cp=(1/2) cd=(1/8), rem=(1/8)
* HV:0:Note:G5 [4] {} cp=(5/8) cd=(1/8), rem=(1/8)
* 0:Barline:Bar:/ 5 * Context:Bar:/ 5
* HV:0:Note:F5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
* HV:0:Note:A5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
* HV:0:Note:F5 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
* HV:0:Note:A5 [4] {} cp=(1/2) cd=(1/8), rem=(1/8)
* HV:0:Note:D5 [4] {} cp=(5/8) cd=(1/8), rem=(1/8)
* 0:Barline:Bar:/ 6 * Context:Bar:/ 6
* HV:0:Note:G5 [3](1){} cp=(0/1) cd=(3/8), rem=(3/8)
* HV:0:Note:E5 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)

```

* HV:0:Note:F5 [4] {} cp=(1/2) cd=(1/8), rem=(1/8)
 * HV:0:Note:G5 [4] {} cp=(5/8) cd=(1/8), rem=(1/8)
 * 0:Barline:Bar:/ 7 * Context:Bar:/ 7
 * HV:0:Note:F6 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
 * HV:0:Note:E6 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:D6 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
 * HV:0:Note:E6 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:C6 [4] {} cp=(1/2) cd=(1/8), rem=(1/8)
 * HV:0:Note:A5 [4] {} cp=(5/8) cd=(1/8), rem=(1/8)
 * 0:Barline:Bar:/ 8 * Context:Bar:/ 8
 * HV:0:Note:G5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
 * HV:0:Note:E5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:A5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
 * HV:0:Note:D5 [3](1){} cp=(3/8) cd=(3/8), rem=(3/8)
 * 0:Barline:Bar:/ 9 * Context:Bar:/ 9
 * HV:0:Note:A5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
 * HV:0:Note:D6 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:A5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
 * HV:0:Note:D6 [3](1){} cp=(3/8) cd=(3/8), rem=(3/8)
 * 0:Barline:Bar:/ 10 * Context:Bar:/ 10
 * HV:0:Note:C5Y[4] {} cp=(0/1) cd=(1/8), rem=(1/8)
 * HV:0:Note:A5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:B5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
 * HV:0:Note:C5Y[4] {} cp=(3/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:A5 [4] {} cp=(1/2) cd=(1/8), rem=(1/8)
 * HV:0:Note:G5 [4] {} cp=(5/8) cd=(1/8), rem=(1/8)
 * 0:Barline:Bar:/ 11 * Context:Bar:/ 11
 * HV:0:Note:A5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
 * HV:0:Note:D6 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:A5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
 * HV:0:Note:D6 [3](1){} cp=(3/8) cd=(3/8), rem=(3/8)
 * 0:Barline:Bar:/ 12 * Context:Bar:/ 12
 * HV:0:Note:C5Y[4] {} cp=(0/1) cd=(1/8), rem=(1/8)
 * HV:0:Note:A5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:B5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
 * HV:0:Note:C5Y[3] {} cp=(3/8) cd=(1/4), rem=(1/4)
 * HV:0:Note:B5 [4] {} cp=(5/8) cd=(1/8), rem=(1/8)
 * 0:Barline:Bar:/ 13 * Context:Bar:/ 13
 * HV:0:Note:A5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
 * HV:0:Note:D6 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:A5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
 * HV:0:Note:D6 [3](1){} cp=(3/8) cd=(3/8), rem=(3/8)
 * 0:Barline:Bar:/ 14 * Context:Bar:/ 14

* HV:0:Note:C5Y[4] {} cp=(0/1) cd=(1/8), rem=(1/8)
 * HV:0:Note:A5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:B5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
 * HV:0:Note:C5Y[4] {} cp=(3/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:D6 [4] {} cp=(1/2) cd=(1/8), rem=(1/8)
 * HV:0:Note:E6 [4] {} cp=(5/8) cd=(1/8), rem=(1/8)
 * 0:Barline:Bar:/ 15 * Context:Bar:/ 15
 * HV:0:Note:F6 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
 * HV:0:Note:E6 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:D6 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
 * HV:0:Note:E6 [4] {} cp=(3/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:C6 [4] {} cp=(1/2) cd=(1/8), rem=(1/8)
 * HV:0:Note:A5 [4] {} cp=(5/8) cd=(1/8), rem=(1/8)
 * 0:Barline:Bar:/ 16 * Context:Bar:/ 16
 * HV:0:Note:G5 [4] {} cp=(0/1) cd=(1/8), rem=(1/8)
 * HV:0:Note:E5 [4] {} cp=(1/8) cd=(1/8), rem=(1/8)
 * HV:0:Note:A5 [4] {} cp=(1/4) cd=(1/8), rem=(1/8)
 * HV:0:Note:D5 [3](1){} cp=(3/8) cd=(3/8), rem=(3/8)
 * 0:Barline:Bar::/I 17 * Context:Bar::/I 17]

Excerpt from Concerto in G major for flute and piano by C.P.E.

Bach

Fig. 4



CPNView Score : from Flute Concerto by C.P.E Bach

Fig. 4a

```

* 0:Clef:TREBLE * Context:TREBLE
* 0:Keysig:ks:#F * Context:ks:#F
* 0:TimeSig:(4,4) * Context:(4,4)
* HV:0:** Rest:[2] {} cp=(0/1) cd=(1/2), rem=(1/2)
* HV:0:Note:F6 [5] {} cp=(1/2) cd=(1/16), rem=(1/16)
* HV:0:Note:D7 [5] {} cp=(9/16) cd=(1/16), rem=(1/16)
* HV:0:Note:A6 [5] {} cp=(5/8) cd=(1/16), rem=(1/16)
* HV:0:Note:F6 [5] {} cp=(11/16) cd=(1/16), rem=(1/16)
* HV:0:Note:D6 [5] {} cp=(3/4) cd=(1/16), rem=(1/16)
* HV:0:Note:C6 [5] {} cp=(13/16) cd=(1/16), rem=(1/16)
* HV:0:Note:B5 [5] {} cp=(7/8) cd=(1/16), rem=(1/16)
* HV:0:Note:A5 [5] {} cp=(15/16) cd=(1/16), rem=(1/16)
* 0:Barline:Bar:/ 2 * Context:Bar:/ 2
* HV:0:Note:B5 [5] {} cp=(0/1) cd=(1/16), rem=(1/16)
* HV:0:Note:D6 [5] {} cp=(1/16) cd=(1/16), rem=(1/16)
* HV:0:Note:D7 [5] {} cp=(1/8) cd=(1/16), rem=(1/16)
* HV:0:Note:D6 [5] {} cp=(3/16) cd=(1/16), rem=(1/16)
* HV:0:Note:C6 [5] {} cp=(1/4) cd=(1/16), rem=(1/16)
* HV:0:Note:D6 [5] {} cp=(5/16) cd=(1/16), rem=(1/16)
* HV:0:Note:C7 [5] {} cp=(3/8) cd=(1/16), rem=(1/16)
* HV:0:Note:D6 [5] {} cp=(7/16) cd=(1/16), rem=(1/16)
* HV:0:Note:G5#[5] {} cp=(1/2) cd=(1/16), rem=(1/16)
* HV:0:Note:D6 [5] {} cp=(9/16) cd=(1/16), rem=(1/16)
* HV:0:Note:B6 [5] {} cp=(5/8) cd=(1/16), rem=(1/16)
* HV:0:Note:D6 [5] {} cp=(11/16) cd=(1/16), rem=(1/16)
* HV:0:Note:F5 [5] {} cp=(3/4) cd=(1/16), rem=(1/16)
* HV:0:Note:D6 [5] {} cp=(13/16) cd=(1/16), rem=(1/16)
* HV:0:Note:A6 [5] {} cp=(7/8) cd=(1/16), rem=(1/16)
* HV:0:Note:D6 [5] {} cp=(15/16) cd=(1/16), rem=(1/16)
* 0:Barline:Bar:/ 3 * Context:Bar:/ 3
* HV:0:Note:E5 [5] {} cp=(0/1) cd=(1/16), rem=(1/16)
* HV:0:Note:D6 [5] {} cp=(1/16) cd=(1/16), rem=(1/16)
* HV:0:Note:G6#[5] {} cp=(1/8) cd=(1/16), rem=(1/16)

```

* HV:0:Note:D6 [5] {} cp=(3/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:E5 [5] {} cp=(1/4) cd=(1/16), rem=(1/16)
 * HV:0:Note:D6 [5] {} cp=(5/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:F5Y[5] {} cp=(3/8) cd=(1/16), rem=(1/16)
 * HV:0:Note:D6 [5] {} cp=(7/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:E5 [5] {} cp=(1/2) cd=(1/16), rem=(1/16)
 * HV:0:Note:C6 [5] {} cp=(9/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:E6 [5] {} cp=(5/8) cd=(1/16), rem=(1/16)
 * HV:0:Note:C6 [5] {} cp=(11/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:E5 [5] {} cp=(3/4) cd=(1/16), rem=(1/16)
 * HV:0:Note:B5 [5] {} cp=(13/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:D6 [5] {} cp=(7/8) cd=(1/16), rem=(1/16)
 * HV:0:Note:B5 [5] {} cp=(15/16) cd=(1/16), rem=(1/16)
 * 0:Barline:Bar:/ 4 * Context:Bar:/ 4
 * HV:0:Note:C6 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
 * HV:0:** Rest:[3] {} cp=(1/4) cd=(1/4), rem=(1/4)
 * HV:0:Note:C6 [5] {} cp=(1/2) cd=(1/16), rem=(1/16)
 * HV:0:Note:A6 [5] {} cp=(9/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:E6 [5] {} cp=(5/8) cd=(1/16), rem=(1/16)
 * HV:0:Note:C6 [5] {} cp=(11/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:A5 [5] {} cp=(3/4) cd=(1/16), rem=(1/16)
 * HV:0:Note:C6 [5] {} cp=(13/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:E6 [5] {} cp=(7/8) cd=(1/16), rem=(1/16)
 * HV:0:Note:A6 [5] {} cp=(15/16) cd=(1/16), rem=(1/16)
 * 0:Barline:Bar:/ 5 * Context:Bar:/ 5
 * HV:0:Note:A5 [5] {} cp=(0/1) cd=(1/16), rem=(1/16)
 * HV:0:Note:C6 [5] {} cp=(1/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:C7 [5] {} cp=(1/8) cd=(1/16), rem=(1/16)
 * HV:0:Note:C6 [5] {} cp=(3/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:G5 [5] {} cp=(1/4) cd=(1/16), rem=(1/16)
 * HV:0:Note:C6 [5] {} cp=(5/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:B6 [5] {} cp=(3/8) cd=(1/16), rem=(1/16)
 * HV:0:Note:C6 [5] {} cp=(7/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:F5 [5] {} cp=(1/2) cd=(1/16), rem=(1/16)
 * HV:0:Note:C6 [5] {} cp=(9/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:A6 [5] {} cp=(5/8) cd=(1/16), rem=(1/16)
 * HV:0:Note:C6 [5] {} cp=(11/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:E5 [5] {} cp=(3/4) cd=(1/16), rem=(1/16)
 * HV:0:Note:C6 [5] {} cp=(13/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:G6 [5] {} cp=(7/8) cd=(1/16), rem=(1/16)
 * HV:0:Note:C6 [5] {} cp=(15/16) cd=(1/16), rem=(1/16)
 * 0:Barline:Bar:/ 6 * Context:Bar:/ 6
 * HV:0:Note:D5 [5] {} cp=(0/1) cd=(1/16), rem=(1/16)

* HV:0:Note:C6 [5] {} cp=(1/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:F6 [5] {} cp=(1/8) cd=(1/16), rem=(1/16)
 * HV:0:Note:C6 [5] {} cp=(3/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:D5 [5] {} cp=(1/4) cd=(1/16), rem=(1/16)
 * HV:0:Note:C6 [5] {} cp=(5/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:E6 [5] {} cp=(3/8) cd=(1/16), rem=(1/16)
 * HV:0:Note:C6 [5] {} cp=(7/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:D5 [5] {} cp=(1/2) cd=(1/16), rem=(1/16)
 * HV:0:Note:B5 [5] {} cp=(9/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:D6 [5] {} cp=(5/8) cd=(1/16), rem=(1/16)
 * HV:0:Note:B5 [5] {} cp=(11/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:D5 [5] {} cp=(3/4) cd=(1/16), rem=(1/16)
 * HV:0:Note:A5 [5] {} cp=(13/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:C6 [5] {} cp=(7/8) cd=(1/16), rem=(1/16)
 * HV:0:Note:A5 [5] {} cp=(15/16) cd=(1/16), rem=(1/16)
 * 0:Barline:Bar:/ 7 * Context:Bar:/ 7
 * HV:0:Note:B5 [3] {} cp=(0/1) cd=(1/4), rem=(1/4)
 * HV:0:** Rest:[3] {} cp=(1/4) cd=(1/4), rem=(1/4)
 * HV:0:Note:B5 [5] {} cp=(1/2) cd=(1/16), rem=(1/16)
 * HV:0:Note:G6 [5] {} cp=(9/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:D6 [5] {} cp=(5/8) cd=(1/16), rem=(1/16)
 * HV:0:Note:B5 [5] {} cp=(11/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:G5 [5] {} cp=(3/4) cd=(1/16), rem=(1/16)
 * HV:0:Note:B6 [5] {} cp=(13/16) cd=(1/16), rem=(1/16)
 * HV:0:Note:G6 [5] {} cp=(7/8) cd=(1/16), rem=(1/16)
 * HV:0:Note:D6 [5] {} cp=(15/16) cd=(1/16), rem=(1/16)
 * 0:Barline:Bar:/ 8 * Context:Bar:/ 8

Appendix 3 : Example of an Enigma file

“The Gander in the Pratie Hole” – see appendix 2 for notation and CPNView score

ENIGMA TRANSPORTABLE FILE

```
^header
^01 "Finale(R) 3.0 Copyright (c) 1993 Coda Music Technology"
^02 0 0 0 0
^03 0 0 0
^04 ""
^05 50332184
^06 "FIN"
^07 "MAC"
^08 50332184
^09 50332184
^10 0 0 0 0 0 0 0 0 0 0 0 0
^11 0 0 0
^12 ""
^13 50397967
^14 "FIN"
^15 "WIN"
^16 50332672
^17 50332421
^18 0 0 0 0 0 0 0 0 0 0 0 0
^19 0 0 0
^20 ""
^21 0
^22 ""
^23 ""
^24 0
^25 0
^26 0 0 0 0 0 0 0 0 0 0 0 0
^27 ""
^28 ""
^29 ""
^30 ""
```

```
^others
^01(65534) 0 0 4 1024 0 0
^04(65534) -72 -64 11 12 0 1
^05(65534) 0 0 0 144 0 0
^06(65534) 61 207 250 119 87 110
^07(65534) 98 35 220 186 46 106
^08(65534) 74 221 227 183 238 206
^09(65534) 228 197 168 244 0 45
^10(65534) 103 251 240 2 1 1
^11(65534) 110 98 35 186 220 215
^12(65534) 1 1 1 1 1 0
^13(65534) 4 1024 75 -8 24 24
```

[illegible]

$^{40}(65534)$ 0 0 0 0 0
 $^{40}(65534)$ 0 0 0 0 0
 $^{40}(65534)$ 0 0 0 0 0
 $^{40}(65534)$ 0 0 0 0 0
 $^{40}(65534)$ 0 0 0 0 0
 $^{40}(65534)$ 0 0 0 0 0
 $^{40}(65534)$ 0 0 0 0 0
 $^{40}(65534)$ 0 0 0 0 0
 $^{40}(65534)$ 0 0 0 0 0
 $^{40}(65534)$ 0 0 0 0 0
 $^{41}(65534)$ 0 0 0 -24 -48 0
 $^{42}(65534)$ 213 243 124 124 213 246
 $^{43}(65534)$ 212 199 215 24 12 24
 $^{44}(65534)$ 0 0 0 7 1 2
 $^{45}(65534)$ 0 0 0 0 0
 $^{50}(65534)$ 0 0 0 0 8 -8
 $^{51}(65534)$ 0 -6 0 -6 0 8
 $^{56}(65534)$ 48 -12 24 -12 515 8
 $^{58}(65534)$ 0 18 0 18 111 111
 $^{59}(65534)$ 111 6 6 8 8 0
 $^{60}(65534)$ 0 0 0 0 0
 $^{61}(65534)$ 0 0 0 0 0
 $^{62}(65534)$ -1 -2500 0 416 0 774
 $^{63}(65534)$ 0 416 0 625 -1 -625
 $^{64}(65534)$ 0 111 1 -11072 0 111
 $^{65}(65534)$ 0 128 1 -11072 1 -11072
 $^{66}(65534)$ -72 -64 11 12 0 1
 $^{An(3)}$ -2 -2 -2 -2 -2 -2
 $^{An(3)}$ -2 26492 0 28090 8 28334
 $^{Ap(3)}$ 2 2 2 2 2 2
 $^{Ap(3)}$ 2 0 0 0 0 2
 $^{BR(8)}$ 0 0 0 0 0 1024
 $^{BR(8)}$ 0 0 0 0 0 0
 $^{BR(16)}$ 0 0 0 0 0 1024
 $^{BR(16)}$ 0 0 0 0 0 0
 $^{DA(0)}$ 72 0 0 0 0 0
 $^{DA(96)}$ 36 0 0 0 0 0
 $^{DA(128)}$ 30 0 0 0 0 0
 $^{DA(170)}$ 36 0 0 0 0 0
 $^{DA(171)}$ 36 0 0 0 0 0
 $^{DA(192)}$ 42 0 0 0 0 0
 $^{DA(256)}$ 42 0 0 0 0 0
 $^{DA(341)}$ 48 0 0 0 0 0
 $^{DA(342)}$ 48 0 0 0 0 0
 $^{DA(384)}$ 60 0 0 0 0 0
 $^{DA(512)}$ 60 0 0 0 0 0
 $^{DA(682)}$ 72 0 0 0 0 0
 $^{DA(683)}$ 72 0 0 0 0 0
 $^{DA(768)}$ 84 0 0 0 0 0
 $^{DA(1024)}$ 84 0 0 0 0 0
 $^{DA(1365)}$ 96 0 0 0 0 0
 $^{DA(1366)}$ 96 0 0 0 0 0
 $^{DA(1536)}$ 108 0 0 0 0 0
 $^{DA(2048)}$ 120 0 0 0 0 0

^DA(3072) 144 0 0 0 0 0
 ^DA(4096) 168 0 0 0 0 0
 ^DO(1) 2 0 0 0 0 2048
 ^DO(2) 3 0 0 0 0 2048
 ^DO(3) 4 0 0 0 0 2048
 ^DO(4) 5 0 0 0 0 2048
 ^DO(5) 6 0 0 0 0 2048
 ^DO(6) 7 0 0 0 0 2048
 ^DO(7) 8 0 1 0 0 10242
 ^DO(8) 10 0 2 0 0 10242
 ^DO(9) 12 0 0 0 0 2048
 ^DO(10) 13 0 0 0 0 2048
 ^DO(11) 14 0 0 0 0 2304
 ^DO(12) 15 0 0 0 0 2048
 ^DO(13) 16 0 0 0 0 2048
 ^DO(14) 17 0 0 0 0 2048
 ^DO(15) 18 0 0 0 0 2048
 ^DO(16) 19 0 0 0 0 2048
 ^DO(17) 20 0 0 0 0 2048
 ^DO(18) 21 0 0 0 0 2048
 ^DO(19) 22 0 0 0 0 2048
 ^DO(20) 23 0 0 0 0 2048
 ^DT(1) 6144 0 88 0 0 2
 ^DT(1) "f"
 ^DT(2) 6144 0 75 0 0 2
 ^DT(2) "F"
 ^DT(3) 6144 0 101 0 0 2
 ^DT(3) "Ä"
 ^DT(4) 6144 0 114 0 0 2
 ^DT(4) "i"
 ^DT(5) 6144 0 127 0 0 2
 ^DT(5) "ë"
 ^DT(6) 6144 0 49 0 0 2
 ^DT(6) "p"
 ^DT(7) 6144 0 62 0 0 2
 ^DT(7) "P"
 ^DT(8) 6144 0 36 0 0 2
 ^DT(8) "i"
 ^DT(9) 6144 0 23 0 0 2
 ^DT(9) ", "
 ^DT(10) 6144 0 10 0 0 2
 ^DT(10) "¬"
 ^DT(11) 6144 0 0 0 0 0
 ^DT(11) "S"
 ^DT(12) 6144 0 0 0 0 0
 ^DT(12) "• "
 ^DT(13) 3595 0 40 1024 0 4097
 ^DT(13) "Adagio"
 ^DT(14) 3595 0 80 1024 0 4097
 ^DT(14) "Moderato"
 ^DT(15) 3595 0 120 1024 0 4097
 ^DT(15) "Allegro"
 ^DT(16) 6144 0 0 0 0 0
 ^DT(16) "ö"

^DT(17) 6144 0 12 0 0 3
 ^DT(17) "Ä"
 ^DT(18) 6144 0 -12 0 0 3
 ^DT(18) "x"
 ^DT(19) 3106 2 3 1024 0 12289
 ^DT(19) "accel."
 ^DT(20) 3106 2 4 1024 0 12289
 ^DT(20) "rit."
 ^DT(21) 3083 0 0 0 0 0
 ^DT(21) "A"
 ^DT(22) 3083 0 0 0 0 0
 ^DT(22) "B"
 ^DT(23) 3083 0 0 0 0 0
 ^DT(23) "C"
 ^DT(24) 3083 0 0 0 0 0
 ^DT(24) "D"
 ^EN(1) "plain"
 ^EN(2) "bold"
 ^EN(3) "italic"
 ^EN(4) "underline"
 ^EN(5) "outline"
 ^EN(6) "shadow"
 ^EN(7) "strikeout"
 ^EN(8) "absolute"
 ^FI(1) 0 0 0 0 0 0
 ^FI(1) 0 0 0 0 0 0
 ^FI(2) 0 0 0 0 0 0
 ^FI(2) 0 0 0 0 0 0
 ^FI(2) 0 0 0 0 0 0
 ^FI(3) 1 0 0 0 0 0
 ^FI(3) 0 0 0 0 0 0
 ^FI(3) 0 0 1 4122 0 0
 ^FI(4) 224 0 0 0 0 0
 ^FI(4) 224 0 32 0 0 0
 ^FI(4) 224 0 0 0 0 0
 ^FI(4) 224 0 0 0 0 0
 ^FI(4) 224 0 0 0 0 0
 ^FI(4) 224 0 0 0 0 0
 ^FI(4) 224 0 0 0 0 0
 ^FI(4) 224 0 0 0 0 0
 ^FI(4) 0 0 0 0 0 0
 ^FI(5) 0 0 0 0 0 0
 ^FI(5) 0 0 0 0 0 0
 ^FI(5) 0 0 0 0 0 0
 ^FI(5) 0 0 0 0 0 0
 ^FI(5) 0 0 0 0 0 0
 ^FI(5) 0 0 0 0 0 0
 ^FI(5) 0 0 0 0 0 0
 ^FI(5) 0 0 0 0 0 0
 ^FI(5) 0 0 0 0 0 0
 ^FI(5) 0 0 0 0 0 0
 ^FI(5) 0 0 0 0 0 0
 ^FI(5) 0 0 0 0 0 0
 ^FI(5) 0 0 0 0 0 0
 ^FI(6) 0 0 0 0 0 0
 ^FI(6) 0 0 0 0 0 0
 ^FI(6) 0 0 0 0 0 0
 ^FI(6) 0 0 0 0 0 0

```

^FI(6) 0 0 0 0 0 0
^FI(6) 0 0 0 0 0 0
^FI(6) 0 0 0 0 0 0
^FI(6) 0 0 0 0 0 0
^FI(6) 0 0 0 0 0 0
^FI(7) 0 0 0 0 0 0
^FI(7) 0 0 0 0 0 0
^FI(8) 0 0 0 0 0 0
^FI(8) 0 0 0 0 0 0
^FI(8) 0 0 0 0 0 0
^FI(8) 0 0 0 0 0 0
^FI(8) 0 0 0 0 0 0
^FI(9) 0 0 0 0 0 0
^FI(10) 64 0 1 0 0 4
^FI(11) 36 72 0 1 111 0
^FI(12) 0 18 0 18 0 0
^FN(0) "Petrucci"
^FN(1) "System"
^FN(4) "Times"
^FN(6) "Seville"
^FN(11) "Times New Ro"
^FN(11) "man"
^FN(34) "Arial"
^FR(1) 1 4 0 0
^FR(2) 5 8 0 0
^FR(3) 9 11 0 0
^FR(4) 12 15 0 0
^FR(5) 16 19 0 0
^FR(6) 20 23 0 0
^FR(7) 24 26 0 0
^FR(8) 27 29 0 0
^FR(9) 30 33 0 0
^FR(10) 34 37 0 0
^FR(11) 38 40 0 0
^FR(12) 41 44 0 0
^FR(13) 45 48 0 0
^FR(14) 49 52 0 0
^FR(15) 53 55 0 0
^FR(16) 56 58 0 0
^FS(1) 1 2 296 230 0 0
^FS(2) 2 5 107 158 0 0
^FS(3) 5 8 107 158 0 0
^FS(4) 8 9 314 158 0 0
^GP(65534) 2 0 0 0 0 0
^HS(0) 0 27 3106 0 0 16386
^HS(0) 0 27 3106 0 0 16386
^HS(1) 0 207 6155 0 0 130
^HT(0) "-#"
^HT(0) ""
^HT(0) ""
^HT(0) ""
^HT(0) "-#"
^HT(0) ""
^HT(0) ""

```

^HT(0) ""
 ^HT(1) "Peg Ryan's"
 ^HT(1) ""
 ^HT(1) ""
 ^HT(1) ""
 ^IK(1) -12 -8 -5 0 0 0
 ^IK(2) -12 -9 -5 0 0 0
 ^IK(3) -12 -8 -4 0 0 0
 ^IK(4) -12 -9 -6 0 0 0
 ^IK(5) -12 -8 -5 -2 0 0
 ^IK(6) -12 -9 -5 -2 0 0
 ^IK(7) -12 -7 -5 0 0 0
 ^IK(8) -12 -5 -3 -8 0 0
 ^IK(9) 3 8 10 -2 0 0
 ^IK(10) -12 -8 -4 -2 0 0
 ^IK(11) -12 -9 -6 0 0 0
 ^IK(12) -12 -9 -6 -3 0 0
 ^IK(13) -12 -8 -5 -1 0 0
 ^IK(14) -12 -8 -2 2 -5 0
 ^IK(15) -8 -3 -10 -14 -3 0
 ^IK(16) -12 -8 -5 -2 2 6
 ^IK(17) -12 -8 -5 -2 1 0
 ^IK(18) -4 -8 -12 0 0 0
 ^IK(19) -12 -7 -5 0 0 0
 ^IK(20) -12 -9 -2 -6 0 0
 ^IK(21) -12 -9 -6 -3 0 0
 ^IK(22) 3 9 7 0 0 0
 ^IK(23) 4 9 7 14 0 0
 ^IK(24) 3 9 7 14 0 0
 ^IK(25) 4 2 7 0 0 0
 ^IK(26) -5 -10 0 0 0 0
 ^IK(27) -5 -8 0 0 0 0
 ^IK(28) -5 -9 0 0 0 0
 ^IK(30) -2 -9 -5 0 0 0
 ^IK(31) -8 -5 -3 0 0 0
 ^IK(32) -9 -3 -5 0 0 0
 ^IS(1) 0 0 0 0 0 0
 ^IU(0) 1 0 -80 0 0 0
 ^IV(1) 77 0 0 3106 0 0
 ^IV(1) 97 42 0 3106 0 0
 ^IV(1) 106 72 0 3106 0 0
 ^IV(2) 109 0 0 3106 0 0
 ^IV(2) 105 40 0 3106 0 0
 ^IV(2) 110 51 0 3106 0 0
 ^IV(3) 97 0 0 3106 0 0
 ^IV(3) 117 29 0 3106 0 0
 ^IV(3) 103 57 0 3106 0 0
 ^IV(4) 100 0 0 3106 0 0
 ^IV(4) 105 25 0 3106 0 0
 ^IV(4) 109 31 0 3106 0 0
 ^IV(5) 7 0 0 3106 0 2048
 ^IV(6) 109 0 0 3106 0 0
 ^IV(6) 105 40 0 3106 0 0
 ^IV(6) 110 50 0 3106 0 0

^V(6) 7 80 0 3106 0 2048
 ^V(7) 115 0 0 3106 0 0
 ^V(7) 117 23 0 3106 0 0
 ^V(7) 115 48 0 3106 0 0
 ^V(7) 4 74 0 3106 0 2048
 ^V(8) 77 0 0 3106 0 0
 ^V(8) 97 42 0 3106 0 0
 ^V(8) 106 72 0 3106 0 0
 ^V(8) 6 86 0 3106 0 2048
 ^V(9) 109 0 0 3106 0 0
 ^V(9) 7 41 0 3106 0 2048
 ^V(9) 40 69 0 3106 0 0
 ^V(9) 35 88 12 4608 0 0
 ^V(9) 5 112 0 3106 0 2048
 ^V(9) 41 141 0 3106 0 0
 ^V(10) 97 0 0 3106 0 0
 ^V(10) 117 29 0 3106 0 0
 ^V(10) 103 56 0 3106 0 0
 ^V(10) 7 86 0 3106 0 2048
 ^V(11) 247 0 23 6144 0 0
 ^V(12) 111 0 27 6144 0 0
 ^V(12) 7 23 0 3106 0 2048
 ^V(13) 77 0 0 3106 0 0
 ^V(13) 97 42 0 3106 0 0
 ^V(13) 106 72 0 3106 0 0
 ^V(13) 7 89 0 3106 0 2048
 ^V(14) 9 0 0 3106 0 2048
 ^V(15) 13 0 0 3106 0 2048
 ^V(16) 55 0 0 3106 0 0
 ^V(16) 40 27 0 3106 0 0
 ^V(16) 35 46 16 4608 0 0
 ^V(16) 49 71 0 3106 0 0
 ^V(16) 49 95 0 3106 0 0
 ^V(16) 41 123 0 3106 0 0
 ^V(17) 7 -4 0 3106 0 2048
 ^V(17) 40 27 0 3106 0 0
 ^V(17) 98 50 12 4608 0 0
 ^V(17) 57 76 0 3106 0 0
 ^V(17) 41 106 0 3106 0 0
 ^V(18) 43 0 36 6144 0 0
 ^V(19) 11 0 0 3106 0 2048
 ^V(20) 109 0 0 3106 0 0
 ^V(20) 7 40 0 3106 0 2048
 ^V(20) 40 67 0 3106 0 0
 ^V(20) 98 91 12 4608 0 0
 ^V(20) 53 120 0 3106 0 0
 ^V(20) 41 149 0 3106 0 0
 ^V(21) 100 0 0 3106 0 0
 ^V(21) 105 25 0 3106 0 0
 ^V(21) 109 31 0 3106 0 0
 ^V(21) 7 78 0 3106 0 2048
 ^V(22) 109 0 0 3106 0 0
 ^V(22) 105 40 0 3106 0 0
 ^V(22) 110 50 0 3106 0 0

^V(22) 6 80 0 3106 0 2048
 ^V(23) 9 0 0 2594 0 2048
 ^V(23) 6 0 31 2594 0 2048
 ^V(24) 109 0 0 3106 0 0
 ^V(24) 105 40 0 3106 0 0
 ^V(24) 110 48 0 3106 0 0
 ^V(24) 9 78 0 2594 0 2048
 ^V(24) 6 79 31 2594 0 2048
 ^V(25) 2 0 0 3106 0 2048
 ^V(26) 115 0 0 3106 0 0
 ^V(26) 117 23 0 3106 0 0
 ^V(26) 115 48 0 3106 0 0
 ^V(26) 2 72 0 3106 0 2048
 ^V(27) 77 0 0 3106 0 0
 ^V(28) 109 0 0 3106 0 0
 ^V(29) 77 0 0 3106 0 0
 ^V(29) 7 38 0 3106 0 2048
 ^V(30) 109 0 0 3106 0 0
 ^V(30) 7 40 0 3106 0 2048
 ^V(31) 77 0 0 3106 0 0
 ^V(31) 6 40 0 3106 0 2048
 ^V(32) 109 0 0 3106 0 0
 ^V(32) 6 40 0 3106 0 2048
 ^X(1) 46 6144 0 40 0 1304
 ^X(1) 46 6144 0 -4 16 1
 ^X(1) 0 0 0 0 0 0
 ^X(2) 62 6144 26 26 0 21016
 ^X(2) 62 6144 0 -4 14 0
 ^X(2) 0 0 0 -12 0 0
 ^X(3) 249 6144 0 0 0 20504
 ^X(3) 223 6144 0 0 19 0
 ^X(3) 0 0 0 -22 0 0
 ^X(4) 94 6144 0 0 0 20504
 ^X(4) 118 6144 0 -8 16 0
 ^X(4) 0 0 0 -32 0 0
 ^X(5) 172 6144 0 0 0 20504
 ^X(5) 232 6144 0 -8 16 0
 ^X(5) 0 0 0 -34 0 0
 ^X(6) 45 6144 0 0 0 24
 ^X(6) 45 6144 0 0 14 1
 ^X(6) 0 0 0 0 0 0
 ^X(7) 60 6144 0 0 0 20504
 ^X(7) 248 6144 0 -6 16 0
 ^X(7) 0 0 0 -14 0 0
 ^X(8) 85 6144 0 0 0 20504
 ^X(8) 117 6144 0 0 23 0
 ^X(8) 0 0 0 0 0 0
 ^X(9) 126 6144 0 0 0 4208
 ^X(9) 126 6144 0 0 0 0
 ^X(9) 0 0 0 0 0 0
 ^X(10) 217 6144 0 0 0 4122
 ^X(10) 217 6144 0 0 36 0
 ^X(10) 0 0 0 0 0 0
 ^X(11) 109 6144 0 0 0 4120

^IX(11) 109 6144 0 0 36 0
 ^IX(11) 0 0 0 0 0 0
 ^IX(12) 77 6144 0 0 0 4120
 ^IX(12) 77 6144 0 0 36 0
 ^IX(12) 0 0 0 0 0 0
 ^IX(13) 178 6144 0 0 0 4122
 ^IX(13) 178 6144 0 0 12 1
 ^IX(13) 0 0 0 0 0 0
 ^IX(14) 179 6144 0 0 0 4122
 ^IX(14) 179 6144 0 0 12 0
 ^IX(14) 0 0 0 0 0 0
 ^IX(15) 84 6144 0 0 0 4122
 ^IX(15) 84 6144 0 10 48 0
 ^IX(15) 0 0 0 0 0 0
 ^IX(16) 161 6144 0 0 0 4123
 ^IX(16) 161 6144 0 -48 59 0
 ^IX(16) 0 0 0 0 0 0
 ^IX(17) 42 6144 0 0 0 4123
 ^IX(17) 42 6144 0 0 59 0
 ^IX(17) 0 0 0 0 0 0
 ^IX(18) 103 6144 -256 0 0 1600
 ^IX(18) 103 6144 0 0 0 0
 ^IX(18) 0 0 0 0 0 0
 ^IX(19) 34 6144 0 0 0 4106
 ^IX(19) 34 6144 36 -36 12 0
 ^IX(19) 0 0 0 0 0 0
 ^IX(20) 201 6144 0 0 0 0
 ^IX(20) 201 6144 0 0 0 0
 ^IX(20) 0 0 0 0 0 0
 ^IX(21) 111 6144 0 0 0 24
 ^IX(21) 111 6144 0 0 17 1
 ^IX(21) 0 0 0 0 0 0
 ^IX(22) 79 6144 0 0 0 24
 ^IX(22) 79 6144 0 0 23 1
 ^IX(22) 0 0 0 0 0 0
 ^IX(23) 43 6144 0 0 0 24
 ^IX(23) 43 6144 0 0 23 1
 ^IX(23) 0 0 0 0 0 0
 ^IX(24) 44 6144 0 50 0 5386
 ^IX(24) 44 6144 24 8 36 0
 ^IX(24) 0 0 0 0 0 0
 ^IX(25) 33 6144 0 0 0 16409
 ^IX(25) 33 6144 12 -12 -48 0
 ^IX(25) 0 0 -12 -16 0 0
 ^IX(26) 64 6144 0 0 0 16409
 ^IX(26) 64 6144 14 -16 -48 0
 ^IX(26) 0 0 -8 -20 0 0
 ^IX(27) 190 6144 0 0 0 16409
 ^IX(27) 190 6144 14 -18 -48 0
 ^IX(27) 0 0 -8 -28 0 0
 ^IX(28) 40 6144 0 0 0 0
 ^IX(28) 40 6144 0 0 0 0
 ^IX(28) 0 0 0 0 0 0
 ^IX(29) 41 6144 0 0 0 0

^IX(29) 41 6144 0 0 0 0
 ^IX(29) 0 0 0 0 0 0
 ^KA(3) 0 60 0 1 1 0
 ^KF(1) 0 0 1 0 0 0
 ^KF(2) 0 0 1 0 0 0
 ^KF(3) 24 7 0 2 2 2
 ^KM(3) -2147483648 0 0
 ^KM(3) 0 -2147483647 0
 ^KM(3) 0 0 -2147483646
 ^KM(3) 0 -2147483645 0
 ^KM(3) 0 0 -2147483644
 ^KM(3) 0 0 0
 ^KM(3) -2147483643 0 0
 ^KM(3) 0 -2147483642 0
 ^LA(0) 6 0 0 0 0 3968
 ^LA(1) -6 0 0 0 0 2944
 ^LA(2) 0 0 0 0 0 0
 ^LA(3) 0 0 0 0 0 0
 ^MN(1) 34 9 0 1 999 1
 ^MN(1) 48 10 0 52 36 0
 ^MN(1) 0 0 0 0 0 0
 ^MN(1) 0 0 0 0 0 0
 ^MN(1) 0 0 0 0 0 0
 ^MN(1) 0 0 0 0 0 0
 ^MN(1) 0 0 0 0 0 -32640
 ^MP(0) 0 32 71 73 16 26
 ^MP(0) 66 48 57 56 70 60
 ^MP(0) 68 52 0 0 0 0
 ^MP(0) 0 0 0 0 0 0
 ^MP(0) 0 0 0 0 0 0
 ^MP(0) 0 0 0 0 0 0
 ^MS(1) 600 1 2 1024 4 16
 ^MS(2) 600 1 2 1024 4 16
 ^MS(3) 600 1 2 1024 4 16
 ^MS(4) 600 1 2 1024 4 16
 ^MS(5) 600 1 2 1024 4 16
 ^MS(6) 600 1 2 1024 4 16
 ^MS(7) 600 1 2 1024 4 16
 ^MS(8) 600 1 2 1024 4 20
 ^MS(9) 600 1 2 1024 4 16
 ^MS(10) 600 1 2 1024 4 16
 ^MS(11) 600 1 2 1024 4 16
 ^MS(12) 600 1 2 1024 4 16
 ^MS(13) 600 1 2 1024 4 16
 ^MS(14) 600 1 2 1024 4 16
 ^MS(15) 600 1 2 1024 4 16
 ^MS(16) 600 1 2 1024 4 20
 ^OC(1) -32768 0 0 0 0 0
 ^OC(2) -32767 0 0 0 0 0
 ^OC(3) -32766 0 0 0 0 0
 ^OC(4) -32765 0 0 0 0 0
 ^OC(5) -32763 0 0 0 0 0
 ^OC(6) -32764 0 0 0 0 0
 ^OC(7) -32762 0 0 0 0 0

^OC(8) -32761 0 0 0 0 0
 ^OC(9) -32760 0 0 0 0 0
 ^OC(10) -32759 0 0 0 0 0
 ^OC(11) -32758 0 0 0 0 0
 ^OC(12) -32757 0 0 0 0 0
 ^OC(13) -32756 0 0 0 0 0
 ^OC(14) -32755 0 0 0 0 0
 ^On(3) 6 2 5 1 4 0
 ^On(3) 3 9922 0 22056 126 -1494
 ^Op(3) 3 0 4 1 5 2
 ^Op(3) 6 0 0 0 0 0
 ^PL(1) 0 0 0 0 0 0
 ^PL(1) 20565 0 0 0 0 0
 ^PO(1) -144 216 144 -144 0 0
 ^PS(1) 3168 2448 1 0
 ^RS(1) 0 0 34 12 2 0
 ^RS(2) 0 0 34 12 2 0
 ^RS(3) 0 0 34 12 2 64
 ^RS(4) 0 0 34 12 2 64
 ^RS(5) 0 0 34 12 2 0
 ^RS(6) 0 0 0 24 0 0
 ^RS(7) 0 0 0 24 0 0
 ^RS(8) 0 0 34 12 2 128
 ^RS(9) 0 0 34 12 2 0
 ^RS(10) 0 0 34 12 2 0
 ^RT(1) "D.C. al Fine"
 ^RT(2) "D.S. al Fine"
 ^RT(3) "To Coda #"
 ^RT(4) "D.S. al #"
 ^RT(5) "Fine"
 ^RT(6) "b"
 ^RT(7) "%"
 ^RT(8) "Go To Measur"
 ^RT(8) "e #"
 ^RT(9) "D.S. al Coda"
 ^RT(10) "D.C. al Coda"
 ^SD(1) 1 1 0 0 0 0
 ^SD(2) 2 2 91 0 -6 395
 ^SD(3) 3 3 -4 -311 -84 4
 ^SD(4) 4 4 282 0 -65 435
 ^SD(5) 5 5 288 -436 -36 0
 ^SD(6) 6 6 -4 0 -444 397
 ^SD(7) 7 7 433 0 0 439
 ^SD(8) 8 8 44 0 -44 492
 ^SD(9) 9 9 288 0 0 292
 ^SD(10) 10 10 44 -496 -44 0
 ^SD(11) 11 11 0 0 -288 288
 ^SD(12) 12 12 0 0 -108 144
 ^SD(13) 13 13 25 5 -30 96
 ^SD(14) 14 14 32 -2 0 35
 ^SD(15) 15 15 2 0 -16 55
 ^SD(16) 16 16 20 0 -4 146
 ^SD(17) 17 17 28 0 4 144
 ^SD(18) 18 18 8 -136 -23 18

^SD(19) 19 19 0 -17 -25 132
 ^SD(20) 20 20 22 0 -29 63
 ^SD(21) 21 21 32 -51 -7 0
 ^SD(22) 22 22 19 4 -23 77
 ^SD(23) 23 23 33 -4 4 62
 ^SD(24) 24 24 288 0 0 288
 ^SD(25) 25 25 12 -12 -288 300
 ^SD(26) 26 26 0 4 -576 868
 ^SD(27) 27 27 0 0 -864 1440
 ^SD(28) 28 28 0 0 -864 1440
 ^SS(1) -463 144 0 -199 1 128
 ^SS(2) 0 0 0 -200 2 128
 ^SS(3) 0 0 0 -200 5 128
 ^SS(4) 0 0 0 -200 8 128
 ^SS(5) 0 0 0 -200 0 0
 ^SS(6) 0 0 0 -200 0 0
 ^SS(7) 0 0 0 -200 0 0
 ^SS(8) 0 0 0 -200 0 0
 ^SS(9) 0 0 0 -200 0 0
 ^SS(10) 0 0 0 -200 0 0
 ^SS(11) 0 0 0 -200 0 0
 ^SS(12) 0 0 0 -200 0 0
 ^SS(13) 0 0 0 -200 0 0
 ^SS(14) 0 0 0 -200 0 0
 ^SS(15) 0 0 0 -200 0 0
 ^SS(16) 0 0 0 -200 0 0
 ^SS(17) 0 0 0 -200 0 0
 ^SS(18) 0 0 0 -200 0 0
 ^SS(19) 0 0 0 -200 0 0
 ^SS(20) 0 0 0 -200 0 0
 ^SS(21) 0 0 0 -200 0 0
 ^SS(22) 0 0 0 -200 0 0
 ^SS(23) 0 0 0 -200 0 0
 ^SS(24) 0 0 0 -200 0 0
 ^SS(25) 0 0 0 -200 0 0
 ^TD(1) 26 0 100 0 0 6144
 ^TD(2) 27 0 100 0 0 6144
 ^TD(3) 28 0 100 0 0 6144
 ^Tn(1) 5 1 4 0 3 6
 ^Tn(1) 2 5 0 0 0 0
 ^Tn(2) 0 1 4 0 3 6
 ^Tn(2) 2 5 0 0 0 0
 ^Tp(1) 5 2 6 3 0 4
 ^Tp(1) 1 5 0 0 0 0
 ^Tp(2) 5 2 6 3 0 4
 ^Tp(2) 1 5 0 0 0 0
 ^XS(1) 9 257 3332 1 0 0
 ^XS(2) 11 257 3332 1 0 0
 ^XS(3) 24 257 3332 1 0 0
 ^XS(4) 25 257 3332 1 0 0
 ^oc(1) 18290 24942 25632 20585 24942 28416
 ^oc(2) 16739 28533 29556 26979 8258 24947
 ^oc(2) 29440 0 0 0 0 0
 ^oc(3) 17260 24946 26990 25972 0 0

^oc(4) 18028 30068 25856 0 0 0
 ^oc(5) 19041 31354 8263 30057 29793 29184
 ^oc(6) 18529 28013 28526 25632 20338 26465
 ^oc(6) 28160 0 0 0 0 0
 ^oc(7) 21605 28271 29216 21345 30720 0
 ^oc(8) 21364 29289 28263 8261 28275 25965
 ^oc(8) 25196 25888 12544 0 0 0
 ^oc(9) 21618 28525 25199 28261 0 0
 ^oc(10) 21618 30061 28773 29696 0 0
 ^oc(11) 16993 29555 28527 28160 0 0
 ^oc(12) 18034 25966 25448 8264 28530 28160
 ^oc(13) 20322 28517 0 0 0 0
 ^oc(14) 17256 28521 29216 16737 26739 0
 ^pd(0) 0 0 0 0 0 2211
 ^sL(1) 34304882 34304879 33647735
 ^sL(1) 33714797 33715044 33714796
 ^sL(1) 33583988 34304879 33647735
 ^sL(1) 33714797 33715044 33714796
 ^sL(1) 33583988 34304879 33647735
 ^sL(1) 33714797 33715044 33714796
 ^sL(1) 33583988 33580402 0
 ^sL(2) 34304879 33714797 33977196
 ^sL(2) 33580659 0 33977196
 ^sL(3) 34304879 33714797 33977196
 ^sL(3) 33580659 0 33977196
 ^sL(4) 34304879 33714797 33977196
 ^sL(4) 33580659 0 33977196
 ^sL(5) 34304879 33714797 33977196
 ^sL(5) 33580659 0 33977196
 ^sL(6) 34304879 33714797 33977196
 ^sL(6) 33580659 0 33977196
 ^sL(7) 34304879 33714797 33977196
 ^sL(7) 33580659 0 33977196
 ^sL(8) 34304879 33714797 33647735
 ^sL(8) 33715044 33714796 33583988
 ^sL(8) 34304879 33714797 33647735
 ^sL(8) 33715044 33714796 33583988
 ^sL(8) 34304879 33647735 33714797
 ^sL(8) 33715044 33714796 33714796
 ^sL(8) 33583988 0 33714796
 ^sL(9) 34304879 33714796 33583988
 ^sL(9) 0 33714796 33583988
 ^sL(10) 34304879 33647735 33714797
 ^sL(10) 33715044 33714796 33583988
 ^sL(10) 34304879 33647735 33714797
 ^sL(10) 33715044 33714796 33583988
 ^sL(10) 34304879 33647735 33714797
 ^sL(10) 33715044 33714796 33714796
 ^sL(10) 33583988 0 33714796
 ^sL(11) 34304879 33714796 33583988
 ^sL(11) 0 33714796 33583988
 ^sL(12) 34304879 33647735 33714797
 ^sL(12) 33715044 33714796 33583988
 ^sL(12) 0 33714796 33583988

101

$\text{^sb}(1)$ -2147483648 -2147483648 2147483647
 $\text{^sb}(1)$ 1000 1000 0
 $\text{^sb}(1)$ 288 0 -144
 $\text{^sb}(1)$ 0 2147483647 -2147483648
 $\text{^sb}(1)$ -2147483648 2147483647 1000
 $\text{^sb}(1)$ 1000 0 0
 $\text{^sb}(1)$ 24 768 0
 $\text{^sb}(1)$ 0 0 0
 $\text{^sb}(1)$ 288 0 -144
 $\text{^sb}(1)$ -24 2147483647 -2147483648
 $\text{^sb}(1)$ -2147483648 2147483647 1000
 $\text{^sb}(1)$ 1000 0 0
 $\text{^sb}(1)$ 24 118 0
 $\text{^sb}(1)$ 0 0 0
 $\text{^sb}(1)$ 0 48 144
 $\text{^sb}(1)$ -24 2147483647 -2147483648
 $\text{^sb}(1)$ -2147483648 2147483647 1000
 $\text{^sb}(1)$ 1000 0 0
 $\text{^sb}(1)$ 24 118 0
 $\text{^sb}(1)$ 0 0 0
 $\text{^sb}(1)$ 0 48 0
 $\text{^sb}(2)$ 0 0 0
 $\text{^sb}(2)$ 91 395 -6
 $\text{^sb}(2)$ 1372 1372 1073741842
 $\text{^sb}(2)$ 0 0 0
 $\text{^sb}(2)$ 0 72 68
 $\text{^sb}(2)$ 144 0 72
 $\text{^sb}(2)$ -68 0 72
 $\text{^sb}(3)$ 4 -4 -311
 $\text{^sb}(3)$ -4 4 -84
 $\text{^sb}(3)$ 1094 1094 -1073741762
 $\text{^sb}(3)$ 0 0 0
 $\text{^sb}(3)$ 0 72 60
 $\text{^sb}(3)$ 144 0 72
 $\text{^sb}(3)$ -60 0 72
 $\text{^sb}(4)$ 0 0 0
 $\text{^sb}(4)$ 282 435 -65
 $\text{^sb}(4)$ 1799 1799 557
 $\text{^sb}(4)$ 0 0 0
 $\text{^sb}(4)$ 0 72 -90
 $\text{^sb}(4)$ 144 0 72
 $\text{^sb}(4)$ 90 0 72
 $\text{^sb}(5)$ 0 0 -436
 $\text{^sb}(5)$ 288 0 -36
 $\text{^sb}(5)$ 1813 1813 -2147483083
 $\text{^sb}(5)$ 0 0 0
 $\text{^sb}(5)$ 0 72 72
 $\text{^sb}(5)$ 144 0 72
 $\text{^sb}(5)$ -72 0 72
 $\text{^sb}(6)$ 0 -4 0
 $\text{^sb}(6)$ -4 397 -444
 $\text{^sb}(6)$ 2028 2028 1073742596
 $\text{^sb}(6)$ 0 0 0
 $\text{^sb}(6)$ 0 72 72

$^{sb}(6)$ 144 0 72
 $^{sb}(6)$ -72 0 72
 $^{sb}(7)$ 0 0 0
 $^{sb}(7)$ 433 439 0
 $^{sb}(7)$ 2139 2139 718
 $^{sb}(7)$ 0 0 0
 $^{sb}(7)$ 0 72 72
 $^{sb}(7)$ 144 0 72
 $^{sb}(7)$ -72 0 72
 $^{sb}(8)$ 0 0 0
 $^{sb}(8)$ 0 0 0
 $^{sb}(8)$ 0 0 0
 $^{sb}(8)$ 0 0 -4
 $^{sb}(8)$ -16 0 18
 $^{sb}(8)$ 0 4 16
 $^{sb}(8)$ 0 0 0
 $^{sb}(8)$ 0 0 0
 $^{sb}(8)$ 0 0 0
 $^{sb}(8)$ 0 0 0
 $^{sb}(8)$ 0 0 18
 $^{sb}(8)$ 0 0 0
 $^{sb}(8)$ 356 28 0
 $^{sb}(8)$ 28 356 -28
 $^{sb}(8)$ 989 778 0
 $^{sb}(8)$ 0 0 128
 $^{sb}(8)$ 0 0 18
 $^{sb}(8)$ 0 -360 -36
 $^{sb}(8)$ 360 -36 -36
 $^{sb}(9)$ 0 0 0
 $^{sb}(9)$ 288 292 0
 $^{sb}(9)$ 1000 1000 0
 $^{sb}(9)$ 0 0 292
 $^{sb}(9)$ 288 0 292
 $^{sb}(10)$ 0 0 0
 $^{sb}(10)$ 0 0 0
 $^{sb}(10)$ 0 0 0
 $^{sb}(10)$ 0 0 128
 $^{sb}(10)$ 0 0 18
 $^{sb}(10)$ 0 0 0
 $^{sb}(10)$ 0 0 0
 $^{sb}(10)$ 0 0 0
 $^{sb}(10)$ 0 0 0
 $^{sb}(10)$ 0 0 128
 $^{sb}(10)$ 0 0 18
 $^{sb}(10)$ 0 0 0
 $^{sb}(10)$ -360 28 -360
 $^{sb}(10)$ 28 0 -28
 $^{sb}(10)$ 1000 778 0
 $^{sb}(10)$ 0 0 128
 $^{sb}(10)$ 0 0 18
 $^{sb}(10)$ 0 360 -36
 $^{sb}(10)$ -360 -36 -36
 $^{sb}(11)$ 0 0 0
 $^{sb}(11)$ 0 288 -288

^sb(11) 1000 1000 0
 ^sb(11) 0 0 288
 ^sb(11) -288 0 288
 ^sb(12) 0 0 0
 ^sb(12) 0 144 -108
 ^sb(12) 1000 1000 0
 ^sb(12) 0 0 128
 ^sb(12) 0 0 18
 ^sb(12) 0 144 -108
 ^sb(13) 0 0 5
 ^sb(13) 25 96 -30
 ^sb(13) 1000 1000 0
 ^sb(13) 0 0 4
 ^sb(13) 0 5 25
 ^sb(13) 93 0 958
 ^sb(13) 1000 0 0
 ^sb(13) 0 512 1
 ^sb(13) 0 18 0
 ^sb(13) 92 25 4
 ^sb(13) -12 6 12
 ^sb(13) 94 -14 958
 ^sb(13) 1000 0 0
 ^sb(13) 0 512 2
 ^sb(13) -2 18 0
 ^sb(13) 92 26 4
 ^sb(13) -44 6 -4
 ^sb(13) 96 -30 958
 ^sb(13) 1000 0 0
 ^sb(13) 0 512 2
 ^sb(13) 14 18 0
 ^sb(13) 94 26 0
 ^sb(14) 0 0 -2
 ^sb(14) 32 35 0
 ^sb(14) 335 335 752
 ^sb(14) 0 0 0
 ^sb(14) 0 40 48
 ^sb(14) 74 -26 8
 ^sb(14) -54 -26 8
 ^sb(15) 0 0 0
 ^sb(15) 2 55 -16
 ^sb(15) 250 250 -1073741653
 ^sb(15) 0 0 -220
 ^sb(15) 28 68 53
 ^sb(15) 104 -4 52
 ^sb(15) -77 -4 52
 ^sb(16) 0 0 0
 ^sb(16) 0 0 0
 ^sb(16) 0 0 0
 ^sb(16) 0 0 128
 ^sb(16) 0 0 18
 ^sb(16) 0 0 0
 ^sb(16) 0 0 0
 ^sb(16) 0 0 0
 ^sb(16) 0 0 0

```

^sb(16) 0 0 128
^sb(16) 0 0 18
^sb(16) 0 0 0
^sb(16) 0 0 0
^sb(16) 0 0 0
^sb(16) 0 0 0
^sb(16) 0 0 128
^sb(16) 0 0 0
^sb(16) 0 0 0
^sb(16) 0 0 0
^sb(16) 0 0 0
^sb(16) 0 0 0
^sb(16) 0 0 128
^sb(16) 0 0 0
^sb(16) 0 0 0
^sb(16) 0 0 0
^sb(16) 0 0 0
^sb(16) 0 0 128
^sb(16) 0 0 0
^sb(16) 0 0 0
^sb(16) 0 0 150
^sb(16) 0 0 24
^sb(16) 150 0 521
^sb(16) 333 0 0
^sb(16) 0 128 0
^sb(16) 0 18 0
^sb(16) -288 0 0
^sb(16) 72 0 0
^sb(17) 144 24 0
^sb(17) 24 144 0
^sb(17) 1000 667 0
^sb(17) 0 0 128
^sb(17) 0 0 18
^sb(17) 0 -144 0
^sb(17) 0 -36 0
^sb(18) -144 0 -144
^sb(18) 0 23 -20
^sb(18) 1000 556 0
^sb(18) 0 0 128
^sb(18) 0 0 18
^sb(18) 0 144 0
^sb(18) 23 -36 0
^sb(19) 144 -1 -17
^sb(19) -1 144 -22
^sb(19) 964 677 0
^sb(19) 0 0 128
^sb(19) 0 0 18
^sb(19) 0 -144 0
^sb(19) -23 -31 0
^sb(20) 0 0 0
^sb(20) 22 63 -29
^sb(20) 198 198 1073742291
^sb(20) 0 0 0

```

^sb(20) 0 46 152
 ^sb(20) 255 -36 57
 ^sb(20) -101 -36 57
 ^sb(21) 0 0 -51
 ^sb(21) 32 -3 -7
 ^sb(21) 604 604 -2147483294
 ^sb(21) 0 0 5
 ^sb(21) 3 35 25
 ^sb(21) 37 -16 23
 ^sb(21) -33 -16 23
 ^sb(22) 4 0 4
 ^sb(22) 19 77 -23
 ^sb(22) 267 267 266
 ^sb(22) 0 0 0
 ^sb(22) 0 72 -107
 ^sb(22) 144 0 72
 ^sb(22) 107 0 72
 ^sb(23) -4 4 -4
 ^sb(23) 33 62 4
 ^sb(23) 472 472 90
 ^sb(23) 0 0 0
 ^sb(23) 0 28 44
 ^sb(23) 72 9 43
 ^sb(23) -51 9 43
 ^sb(24) 0 0 0
 ^sb(24) 288 288 0
 ^sb(24) 1000 1000 0
 ^sb(24) 0 0 288
 ^sb(24) 288 0 288
 ^sb(25) 0 0 0
 ^sb(25) 0 288 -288
 ^sb(25) 1000 1000 0
 ^sb(25) 0 0 288
 ^sb(25) -288 0 288
 ^sb(26) 4 0 4
 ^sb(26) 0 868 -576
 ^sb(26) 747 993 0
 ^sb(26) 0 0 256
 ^sb(26) 18 0 1156
 ^sb(26) -580 0 1156
 ^sb(27) 0 0 0
 ^sb(27) 0 1440 -864
 ^sb(27) 1000 1000 0
 ^sb(27) 0 0 256
 ^sb(27) 18 0 1440
 ^sb(27) -864 0 1440
 ^sb(28) 0 0 0
 ^sb(28) 0 1440 -864
 ^sb(28) 1000 1000 0
 ^sb(28) 0 0 256
 ^sb(28) 18 0 1440
 ^sb(28) -864 436 -292
 ^sb(28) 436 -292 1008
 ^sb(28) -576 662 986

$\text{sb}(28) \ 0 \ 0 \ 0$
 $\text{sb}(28) \ 256 \ 18 \ 0$
 $\text{sb}(28) \ 864 \ -288 \ 0$

^details

$\text{CL}(0,0) \ 144 \ 0 \ 0 \ 0$
 $\text{GF}(1,1) \ 1 \ 0 \ 0 \ 0 \ 0$
 $\text{GF}(1,2) \ 2 \ 0 \ 0 \ 0 \ 0$
 $\text{GF}(1,3) \ 3 \ 0 \ 0 \ 0 \ 0$
 $\text{GF}(1,4) \ 4 \ 0 \ 0 \ 0 \ 0$
 $\text{GF}(1,5) \ 5 \ 0 \ 0 \ 0 \ 0$
 $\text{GF}(1,6) \ 6 \ 0 \ 0 \ 0 \ 0$
 $\text{GF}(1,7) \ 7 \ 0 \ 0 \ 0 \ 0$
 $\text{GF}(1,8) \ 8 \ 0 \ 0 \ 0 \ 0$
 $\text{GF}(1,9) \ 9 \ 0 \ 0 \ 0 \ 0$
 $\text{GF}(1,10) \ 10 \ 0 \ 0 \ 0 \ 0$
 $\text{GF}(1,11) \ 11 \ 0 \ 0 \ 0 \ 0$
 $\text{GF}(1,12) \ 12 \ 0 \ 0 \ 0 \ 0$
 $\text{GF}(1,13) \ 13 \ 0 \ 0 \ 0 \ 0$
 $\text{GF}(1,14) \ 14 \ 0 \ 0 \ 0 \ 0$
 $\text{GF}(1,15) \ 15 \ 0 \ 0 \ 0 \ 0$
 $\text{GF}(1,16) \ 16 \ 0 \ 0 \ 0 \ 0$
 $\text{KS}(1,1) \ 36 \ 0 \ 0 \ 12 \ 0 \ 8 \ 93 \ -112 \ 1 \ 0$
 $\text{KS}(1,2) \ 35 \ 0 \ 0 \ 12 \ 0 \ 8 \ 93 \ -112 \ 1 \ 0$
 $\text{KS}(1,3) \ 36 \ 35 \ 0 \ 0 \ -8 \ 14 \ 0 \ 0 \ 0 \ 0$
 $\text{KS}(1,4) \ -36 \ 0 \ 0 \ 0 \ 41 \ 120 \ -1 \ -7 \ 41 \ 120$
 $\text{KS}(1,5) \ 36 \ -36 \ 0 \ 0 \ 0 \ -127 \ 0 \ 0 \ 0 \ 126$
 $\text{KS}(1,6) \ 35 \ -36 \ 0 \ 0 \ 1 \ 104 \ 0 \ 0 \ 0 \ 112$
 $\text{KS}(1,7) \ 36 \ 35 \ -36 \ 0 \ 0 \ 10 \ 1 \ 27 \ 0 \ 113$
 $\text{KS}(1,8) \ -36 \ -36 \ 0 \ 0 \ 0 \ 20 \ 0 \ 111 \ 86 \ 28$
 $\text{KS}(1,10) \ 35 \ -36 \ -36 \ 0 \ 0 \ -64 \ 1 \ -113 \ -96 \ 35$
 $\text{KS}(1,12) \ -36 \ -36 \ -36 \ 0 \ 0 \ -128 \ -37 \ -56 \ 64 \ -128$
 $\text{KS}(1,14) \ 35 \ -36 \ -36 \ -36 \ 0 \ 0 \ 0 \ 17 \ 0 \ 10$
 $\text{KS}(1,16) \ -36 \ -36 \ -36 \ -36 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0$
 $\text{KS}(1,18) \ 35 \ -36 \ -36 \ -36 \ -36 \ 0 \ 0 \ 88 \ 0 \ 113$
 $\text{KS}(1,20) \ -36 \ -36 \ -36 \ -36 \ -36 \ 0 \ 0 \ -34 \ 0 \ 1$
 $\text{KS}(1,22) \ 35 \ -36 \ -36 \ -36 \ -36 \ -36 \ 0 \ 0 \ 1 \ -113$
 $\text{KS}(1,24) \ -36 \ -36 \ -36 \ -36 \ -36 \ -36 \ 0 \ 0 \ -32 \ 14$
 $\text{KS}(1,26) \ 35 \ -36 \ -36 \ -36 \ -36 \ -36 \ -36 \ 0 \ 0 \ 10$
 $\text{KS}(1,28) \ -36 \ -36 \ -36 \ -36 \ -36 \ -36 \ -36 \ 0 \ 0 \ 2$
 $\text{KS}(1,30) \ 35 \ -36 \ -36 \ -36 \ -36 \ -36 \ -36 \ -36 \ 0 \ 52$
 $\text{KS}(1,32) \ -36 \ -36 \ -36 \ -36 \ -36 \ -36 \ -36 \ -36 \ 0 \ -128$
 $\text{KS}(1,34) \ -36 \ -36 \ -36 \ -36 \ -36 \ -36 \ -36 \ -36 \ 0 \ 0$
 $\text{KS}(1,36) \ -36 \ -36 \ -36 \ -36 \ -36 \ -36 \ -36 \ 0 \ 0 \ 0$
 $\text{KS}(1,38) \ -36 \ -36 \ -36 \ -36 \ -36 \ -36 \ -36 \ 0 \ 0 \ 2$
 $\text{v1}(0,0) \ -144 \ 0 \ 0 \ 0$
 $\text{v2}(0,0) \ -184 \ 0 \ 0 \ 0$
 $\text{v3}(0,0) \ -224 \ 0 \ 0 \ 0$
 $\text{v4}(0,0) \ -264 \ 0 \ 0 \ 0$
 $\text{v5}(0,0) \ -304 \ 0 \ 0 \ 0$
 $\text{v6}(0,0) \ -344 \ 0 \ 0 \ 0$
 $\text{v7}(0,0) \ -384 \ 0 \ 0 \ 0$

$\wedge\#v8(0,0)$ -424 0 0 0
 $\wedge\#v9(0,0)$ -464 0 0 0
 $\wedge\#v10(0,0)$ -504 0 0 0
 $\wedge\#c1(0,0)$ -144 0 0 0
 $\wedge\#c2(0,0)$ -184 0 0 0
 $\wedge\#c3(0,0)$ -224 0 0 0
 $\wedge\#c4(0,0)$ -264 0 0 0
 $\wedge\#c5(0,0)$ -304 0 0 0
 $\wedge\#c6(0,0)$ -344 0 0 0
 $\wedge\#c7(0,0)$ -384 0 0 0
 $\wedge\#c8(0,0)$ -424 0 0 0
 $\wedge\#c9(0,0)$ -464 0 0 0
 $\wedge\#c10(0,0)$ -504 0 0 0
 $\wedge\#s1(0,0)$ -144 0 0 0
 $\wedge\#s2(0,0)$ -184 0 0 0
 $\wedge\#s3(0,0)$ -224 0 0 0
 $\wedge\#s4(0,0)$ -264 0 0 0
 $\wedge\#s5(0,0)$ -304 0 0 0
 $\wedge\#s6(0,0)$ -344 0 0 0
 $\wedge\#s7(0,0)$ -384 0 0 0
 $\wedge\#s8(0,0)$ -424 0 0 0
 $\wedge\#s9(0,0)$ -464 0 0 0
 $\wedge\#s10(0,0)$ -504 0 0 0

$\wedge$ entries

$\wedge e(1)$ 0 2 512 0 \$C0000800 0 1
 32 \$80000000
 $\wedge e(2)$ 1 3 512 0 \$C0000000 0 1
 -48 \$80000000
 $\wedge e(3)$ 2 4 512 0 \$C0000800 0 1
 -32 \$80000000
 $\wedge e(4)$ 3 5 512 0 \$C0000000 0 1
 -48 \$80000000
 $\wedge e(5)$ 4 6 512 0 \$C0000800 0 1
 32 \$80000000
 $\wedge e(6)$ 5 7 512 0 \$C0000000 0 1
 -48 \$80000000
 $\wedge e(7)$ 6 8 512 0 \$C0000800 0 1
 -32 \$80000000
 $\wedge e(8)$ 7 9 512 0 \$C0000000 0 1
 -48 \$80000000
 $\wedge e(9)$ 8 10 1024 0 \$C0000800 0 1
 0 \$80000000
 $\wedge e(10)$ 9 11 768 0 \$C0000800 0 1
 16 \$80000000
 $\wedge e(11)$ 10 12 256 0 \$C0000000 0 1
 32 \$80000000
 $\wedge e(12)$ 11 13 512 0 \$C0000800 0 1
 16 \$80000000
 $\wedge e(13)$ 12 14 512 0 \$C0000000 0 1
 0 \$80000000
 $\wedge e(14)$ 13 15 512 0 \$C0000800 0 1

```

-32 $800000000
^eE(15) 14 16 512 0 $C0000000 0 1
-48 $800000000
^eE(16) 15 17 512 0 $C0000800 0 1
32 $800000000
^eE(17) 16 18 512 0 $C0000000 0 1
-48 $800000000
^eE(18) 17 19 512 0 $C0000800 0 1
-32 $800000000
^eE(19) 18 20 512 0 $C0000000 0 1
-48 $800000000
^eE(20) 19 21 512 0 $C0000800 0 1
32 $800000000
^eE(21) 20 22 512 0 $C0000000 0 1
-48 $800000000
^eE(22) 21 23 512 0 $C0000800 0 1
-32 $800000000
^eE(23) 22 24 512 0 $C0000000 0 1
-48 $800000000
^eE(24) 23 25 1024 0 $C0000800 0 1
0 $800000000
^eE(25) 24 26 768 0 $C0000800 0 1
16 $800000000
^eE(26) 25 27 256 0 $C0000000 0 1
32 $800000000
^eE(27) 26 28 512 0 $C0000800 0 1
16 $800000000
^eE(28) 27 29 512 0 $C0000000 0 1
0 $800000000
^eE(29) 28 30 1024 0 $C0000800 0 1
0 $800000000
^eE(30) 29 31 512 0 $C0000800 0 1
32 $800000000
^eE(31) 30 32 512 0 $C0000000 0 1
64 $800000000
^eE(32) 31 33 512 0 $C0000800 0 1
32 $800000000
^eE(33) 32 34 512 0 $C0000000 0 1
16 $800000000
^eE(34) 33 35 512 0 $C0000800 0 1
16 $800000000
^eE(35) 34 36 512 0 $C0000000 0 1
0 $800000000
^eE(36) 35 37 512 0 $C0000800 0 1
-32 $800000000
^eE(37) 36 38 512 0 $C0000000 0 1
-48 $800000000
^eE(38) 37 39 1024 0 $C0000800 0 1
0 $800000000
^eE(39) 38 40 768 0 $C0000800 0 1
16 $800000000
^eE(40) 39 41 256 0 $C0000000 0 1
32 $800000000
^eE(41) 40 42 512 0 $C0000800 0 1

```

16 \$80000000
 ^eE(42) 41 43 512 0 \$C0000000 0 1
 0 \$80000000
 ^eE(43) 42 44 512 0 \$C0000800 0 1
 -32 \$80000000
 ^eE(44) 43 45 512 0 \$C0000000 0 1
 -48 \$80000000
 ^eE(45) 44 46 512 0 \$C0000800 0 1
 32 \$80000000
 ^eE(46) 45 47 512 0 \$C0000000 0 1
 64 \$80000000
 ^eE(47) 46 48 512 0 \$C0000800 0 1
 32 \$80000000
 ^eE(48) 47 49 512 0 \$C0000000 0 1
 16 \$80000000
 ^eE(49) 48 50 512 0 \$C0000800 0 1
 16 \$80000000
 ^eE(50) 49 51 512 0 \$C0000000 0 1
 0 \$80000000
 ^eE(51) 50 52 512 0 \$C0000800 0 1
 -32 \$80000000
 ^eE(52) 51 53 512 0 \$C0000000 0 1
 -48 \$80000000
 ^eE(53) 52 54 1024 0 \$C0000800 0 1
 0 \$80000000
 ^eE(54) 53 55 768 0 \$C0000800 0 1
 16 \$80000000
 ^eE(55) 54 56 256 0 \$C0000000 0 1
 32 \$80000000
 ^eE(56) 55 57 512 0 \$C0000800 0 1
 16 \$80000000
 ^eE(57) 56 58 512 0 \$C0000000 0 1
 0 \$80000000
 ^eE(58) 57 0 1024 0 \$C0000800 0 1
 0 \$80000000

^text

^lyrics

Bibliography

Belkin, Alan, and Cindy Grande, 1996. The Development of the Notation Interchange File Format. The Computer Music Journal. 1997, 20 (4), pp.33-43.

Grande, Cindy, 1997. *The Notation Interchange File Format : A Windows - Compliant Approach*. In: E. Selfridge-Field, ed. *Beyond MIDI - A Handbook of Musical Codes*. Massachusetts : MIT Press.

O Maidin, Donncha S., 1995. *A Programmer's Environment for Music Analysis*. Cork.

Schneider, G. Michael, and Judith L. Gersting, 1995. *An Invitation to Computer Science*. Minneapolis / St.Paul : West.

Selfridge-Field, Eleanor, ed., 1997. *Beyond MIDI - The Handbook of Musical Code*. Massachusetts : MIT Press

Selfridge-Field, Eleanor, 1997. *DARMS, Its Dialects, and Its Uses*. In : E. Selfridge-Field, ed. *Beyond MIDI - A Handbook of Musical Codes*. Massachusetts : MIT Press.

Sloan, Donald, 1997. *HyTime and Standard Music Description Language : A Document-Description Approach*. In: E. Selfridge-Field, ed. *Beyond MIDI - A Handbook of Musical Codes*. Massachusetts : MIT Press.

Sloan, Donald, 1992. HyTime / Standard Music Description Language and Computer-Aided Music Research. *Computing in Musicology*. 1992, (8), pp. 127-130.

Turner, Robert C., Timothy A. Douglas and Audrey J. Turner, 1996. *Read me 1st - SGML for Writers and Editors*. New Jersey : Prentice Hall.

Websites:

Music Representation:

Coda Music Technology's Enigma Transportable File Format documentation
<http://www.codamusic.com/Software.html>

Humdrum Kern site maintained by The University of Virginia
<http://www.lib.virginia.edu/dmmc/Music/Humdrum/index.html>

NIFF:
Tauber, James K.
<http://www.jtauber.com/music/encoding/niff/spec/niff-02.html>

NIFF site maintained by the technical coordinator of NIFF, Alan Belkin
<http://mistral.ere.umontreal.ca/~belkina/NIFF.doc.html>

Slomin, David G., at Princeton University
<http://www.cs.princeton.edu/~dgslomin/nifty/paper/index.html>

SMDL & NIFF:
Mounce, Steve, at the University of Bradford, England
<http://www.student.brad.ac.uk/srmounce/encoding.html>

Electronic Journals and Digests:

The Finale Digest, maintained by Sam Houston University
<ftp://ftp.shsu.edu/pub/finale/>

The Finale Forum
<http://www.cmp.net/finale/index.html>

Related Commercial Web Sites:

Musicware's Nightingale site
<http://www.musicwareinc.com/>

Musitek's MIDISCAN Site
<http://www.musitek.com/niff.html>

General Computer Music Sites:

The Computer Music Journal

<http://mitpress.mit.edu/e-journals/Computer-Music-Journal/>

The Stanford University Center for Computer Assisted Research in the Humanities

<http://musedata.stanford.edu/>

The Stanford University Center for Computer Research in Music and Acoustics

<http://ccrma-www.stanford.edu/>